

STRATEGY FOR DATA WAREHOUSE

Data Preparation

- Data Extraction
- Raw Data Analysis
- Data Merging

Data Cleaning

- Data Conversion
- Data Mapping
- Data Integration
- Handling Invalid Data

Data Exploration

- Observe Data Behavior
- Observe Classes Performance
- Observe a Class attribute(s) Performance

Data Mining

- Techniques
 - o Data Modeling
 - o Results
 - o Validation
- Advanced Techniques
 - o Data Modeling
 - o Results
 - o Validation

SYSTEM IMPLEMENTATION

Introduction

- Motivation
- The Need – based on data analysis
- The Advantages

System Specifications

- Hardware
- Software
- Limitations

Project Architecture

- System Architecture
- Milestones

Project Development

- Phase I – Data Mining Algorithms
- Phase II – Advanced Data Mining Algorithms for Machine Learning Integration
- Phase II – Data Visualization

Project Implementation

- Test Application
- Test Results

STRATEGY FOR DATA WAREHOUSE

Data Preparation

- Data Extraction

```
> picknpack <- read.csv("~/workspace/LTB-warehouse/picknpack.csv")
> str(picknpack)
'data.frame': 60561 obs. of 26 variables:
 $ ppDate.ppTime      : Factor w/ 19852 levels "4/11/2016 10:11",...: 313 313 313 314 314 314 315 316 317 317 ...
 $ User.ID            : Factor w/ 24 levels "ALEJANDROM","ALICIAM",...: 5 5 5 5 5 5 5 5 5 5 ...
 $ Quantity           : int  4 4 3 3 3 3 4 4 3 ...
 $ Zone              : Factor w/ 6 levels "","200S","BINSHELF",...: 2 2 2 2 2 2 2 2 2 2 ...
 $ Area              : Factor w/ 5 levels "PIFLOORPP-Phase 1 - Floor Piece Pick",...: 2 2 2 2 2 2 2 2 2 2 ...
 $ Travel.Sequence    : num  1010000000000000 1010000000000000 1010000000000000 1010000000000000 1010000000000000 ...
 $ Storage.Location   : Factor w/ 5072 levels "2110530201","2110530202",...: 2324 2324 2324 2585 2585 2585 2585 3198 3198 3198 ...
 $ aisle             : Factor w/ 19 levels "","211","212",...: 12 12 12 13 13 13 13 14 14 14 ...
 $ bay               : int  53 53 53 13 13 13 13 55 55 55 ...
 $ Level            : Factor w/ 13 levels "","1","2","3",...: 8 8 8 8 8 8 8 8 9 9 9 ...
 $ position          : Factor w/ 9 levels "","1","2","A",...: 1 1 1 1 1 1 1 1 1 ...
 $ OrderDate.OrderTime : Factor w/ 2371 levels "3/28/2016 6:48",...: 857 857 857 857 857 857 857 857 857 ...
 $ order.Number       : int  804062448 804062448 804062448 804062448 804062448 804062448 804062448 804062448 ...
 $ order.Line        : int  70 70 70 60 60 60 60 100 100 100 ...
 $ Part.Number        : Factor w/ 4597 levels "109B6400","14001-i7",...: 4496 4496 4496 4422 4422 4422 4422 4425 4425 ...
 $ Work.Reference     : Factor w/ 58055 levels "WRR0407475","WRR0407477",...: 58 58 58 54 54 54 68 69 69 ...
 $ Shipment.ID        : Factor w/ 35872 levels "SID2284248","SID2285926",...: 21 21 21 21 21 21 21 21 21 ...
 $ shipped_date.shipped_time : Factor w/ 1902 levels "4/11/2016 11:20",...: 47 47 47 47 47 47 47 47 47 ...
 $ Carrier.Code       : Factor w/ 57 levels "AAFES PRIVATE FLEET",...: 35 35 35 35 35 35 35 35 35 ...
 $ car_type           : Factor w/ 3 levels "EXP","GROUND",...: 3 3 3 3 3 3 3 3 3 ...
 $ Bill.To.Customer   : Factor w/ 236 levels "100151","100207",...: 73 73 73 73 73 73 73 73 73 ...
 $ Address.Name       : Factor w/ 236 levels "A HEAVENLY CELEBRATION",...: 10 10 10 10 10 10 10 10 10 ...
 $ early_ship_date.early_ship_time : Factor w/ 162 levels "","1/12/2016 0:00",...: 116 116 116 116 116 116 116 116 116 ...
 $ late_ship_date.late_ship_time : Factor w/ 104 levels "","10/21/2016 0:00",...: 88 88 88 88 88 88 88 88 88 ...
 $ ext_date.ext_time  : Factor w/ 3 levels "","4/13/2016 0:00",...: 1 1 1 1 1 1 1 1 1 ...
 $ calc_pri          : int  9 9 9 9 9 9 9 9 9 ...
```

- Raw Data Analysis

```
> summary(picknpack)
ppDate.ppTime      User.ID      Quantity      Zone
5/20/2016 11:22: 244      NOELC      : 8381      Min.: 1.000000      Zone: 1
5/20/2016 11:21: 226      HECTORJR : 8134      1st Qu.: 1.000000      200S : 31427
5/20/2016 11:26: 226      STEVEM   : 7922      Median: 1.000000      BINSHELF: 5913
5/20/2016 11:23: 223      STEPHANIAG: 6046      Mean: 2.042205      FLOOR : 2159
5/20/2016 11:24: 219      AUDELIAZ : 4991      3rd Qu.: 2.000000      FLOW123: 19432
5/20/2016 11:25: 210      NADIAS   : 3336      Max.: 213.000000      TOWER : 1629
(Other): 59213      (Other): 21751

Area      Travel.Sequence      Storage.Location
Min.: 2159      Min.: 2110640201      Min.: 709
1st Qu.: 47795      1st Qu.: 1010000000000000      1st Qu.: 584
Median: 1      Median: 1010000000000000      Median: 307
Mean: 3064      Mean: 115277902887000      Mean: 251
3rd Qu.: 7542      3rd Qu.: 1010000000000000      3rd Qu.: 204
Max.: 3050000000000000      Max.: 3050000000000000      Max.: 175
(Other): 58331

aisle      bay      Level      position      OrderDate.OrderTime      Order.Number
FL001 : 8046      Min.: 1.00000      B : 11029      : 51584      5/18/2016 10:41: 1504      Min.: 804029477
FL002 : 6197      1st Qu.: 32.00000      A : 10843      1 : 1829      4/13/2016 13:43: 328      1st Qu.: 804120039
FL216 : 5596      Median: 56.00000      C : 9978      A : 1289      4/22/2016 5:42: 317      Median: 804170291
FL003 : 5189      Mean: 57.59347      D : 6514      2 : 1235      4/22/2016 5:47: 302      Mean: 804169327
FL217 : 4548      3rd Qu.: 76.00000      F : 4911      E : 1031      5/24/2016 14:15: 189      3rd Qu.: 804219127
FL220 : 4021      Max.: 1030.00000      E : 4520      B : 1021      5/24/2016 14:13: 187      Max.: 804276047
(Other): 26964      NA's : 1630      (Other): 12766      (Other): 2572      (Other): 57734

Order.Line      Part.Number      Work.Reference      Shipment.ID      shipped_date.shipped_time
Min.: 10.0000      5114325: 537      WRR0652985: 11      SID2327437: 1501      5/23/2016 12:22: 1501
1st Qu.: 10.0000      5106253: 426      WRR0758414: 10      SID2304272: 564      4/29/2016 13:41: 564
Median: 10.0000      5173172: 426      WRR0407475: 8      SID2320146: 386      5/13/2016 14:39: 388
Mean: 76.6632      5168060: 418      WRR0407489: 8      SID2304271: 315      4/29/2016 14:08: 316
3rd Qu.: 30.0000      5081298: 337      WRR0510657: 8      SID2298118: 292      4/22/2016 12:52: 305
Max.: 3870.0000      5050669: 279      WRR0627485: 8      SID2307555: 206      5/31/2016 18:15: 209
(Other): 58138      (Other): 60508      (Other): 57297      (Other): 57278

Carrier.Code      car_type      Bill.To.Customer      Address.Name
UPS GROUND : 40191      EXP : 2599      151638 : 12140      MFKASA.COM - DROP SHIP : 12140
FEDEX HOME DELIVERY : 3941      GROUND: 48529      155438 : 6654      PFALTZGRAFF.COM DROP SHIP: 6654
USPS PRIORITY MAIL : 2712      TL-LTL: 9433      155559 : 5506      KOHLS.COM -WHOLESALE DS : 5506
EMPLOYEE SALE : 1577      140950 : 5317      WAYFAIR LLC : 5317
FED EX GROUND : 1332      104827 : 3224      DILLARD DEPARTMENT STORES: 3224
Overflow to Consolidator: 1302      105763 : 3204      MACY S.COM FONTANA : 3204
(Other): 9506      (Other): 24516      (Other): 24516

early_ship_date.early_ship_time      late_ship_date.late_ship_time      ext_date.ext_time      calc_pri
5/17/2016 0:00: 2467      : 50953      : 60532      Min.: 0.000000
5/30/2016 0:00: 2046      5/6/2016 0:00 : 1143      4/13/2016 0:00: 25      1st Qu.: 0.000000
5/9/2016 0:00 : 1606      6/11/2016 0:00: 869      5/6/2016 0:00 : 4      Median: 9.000000
5/23/2016 0:00: 1438      10/5/2016 0:00: 322      Mean: 8.613761
5/2/2016 0:00 : 1419      5/28/2016 0:00: 306      3rd Qu.: 9.000000
4/11/2016 0:00: 1338      5/14/2016 0:00: 282      Max.: 9.000000
(Other): 50247      (Other): 6686
```

- Data Merging

Data Cleaning

- Data Attributes

```
> names(picknpack)
[1] "ppDate.ppTime"      "User.ID"            "Quantity"
[4] "Zone"               "Area"               "Travel.Sequence"
[7] "Storage.Location"   "aisle"              "bay"
[10] "Level"              "position"            "OrderDate.OrderTime"
[13] "Order.Number"       "Order.Line"         "Part.Number"
[16] "Work.Reference"     "Shipment.ID"        "shipped_date.shipped_time"
[19] "Carrier.Code"       "car_type"           "Bill.To.Customer"
[22] "Address.Name"       "early_ship_date.early_ship_time" "late_ship_date.late_ship_time"
[25] "ext_date.ext_time"  "calc_pri"
```

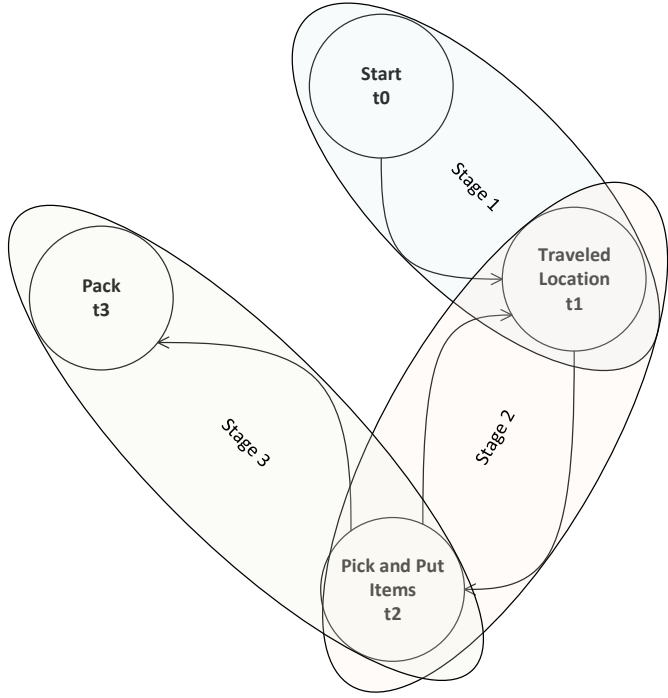
- Conversion
- Mapping

- Data Integration

```
> df <- read.csv("~/workspace/LTB-warehouse/datar1ts.csv")
> str(df)
'data.frame':   60561 obs. of  47 variables:
 $ pm      : int  4 4 4 4 4 4 4 4 4 4 ...
 $ pd      : int  11 11 11 11 11 11 11 11 11 ...
 $ py      : int  2016 2016 2016 2016 2016 2016 2016 2016 2016 ...
 $ ppTime   : num  6.09 6.09 6.09 6.12 6.12 6.12 6.13 6.14 6.15 ...
 $ UserID   : int  1 1 1 1 1 1 1 1 1 1 ...
 $ Quantity : int  4 4 4 3 3 3 3 4 4 3 ...
 $ Zone     : int  1 1 1 1 1 1 1 1 1 1 ...
 $ Area     : int  1 1 1 1 1 1 1 1 1 1 ...
 $ TravelSequence : num  1.01e+14 1.01e+14 1.01e+14 1.01e+14 1.01e+14 1.01e+14 ...
 $ StorageLocation : int  1 1 1 2 2 2 2 3 3 3 ...
 $ aisle    : int  1 1 1 2 2 2 2 3 3 3 ...
 $ bay      : int  53 53 53 13 13 13 13 55 55 55 ...
 $ Level    : int  1 1 1 1 1 1 1 2 2 2 ...
 $ position : int  1 1 1 1 1 1 1 1 1 1 ...
 $ Om       : int  4 4 4 4 4 4 4 4 4 4 ...
 $ Od       : int  6 6 6 6 6 6 6 6 6 6 ...
 $ Oy       : int  2016 2016 2016 2016 2016 2016 2016 2016 2016 ...
 $ Ordertime : num  8.11 8.11 8.11 8.11 8.11 8.11 8.11 8.11 8.11 ...
 $ OrderNumber : int  1 1 1 1 1 1 1 1 1 1 ...
 $ OrderLine  : int  70 70 70 60 60 60 60 100 100 100 ...
 $ PartNumber : int  1 1 1 2 2 2 2 3 3 3 ...
 $ WorkReference : int  1 1 1 2 2 2 3 4 4 4 ...
 $ ShipmentID  : int  1 1 1 1 1 1 1 1 1 1 ...
 $ Sm         : int  4 4 4 4 4 4 4 4 4 4 ...
 $ Sd         : int  12 12 12 12 12 12 12 12 12 ...
 $ Sy         : int  2016 2016 2016 2016 2016 2016 2016 2016 2016 ...
 $ shipped_time : num  12.6 12.6 12.6 12.6 12.6 12.6 ...
 $ CarrierCode : int  1 1 1 1 1 1 1 1 1 1 ...
 $ car_type    : int  1 1 1 1 1 1 1 1 1 1 ...
 $ Bill.ToCustomer : int  135930 135930 135930 135930 135930 135930 135930 135930 ...
 $ AddressName : int  1 1 1 1 1 1 1 1 1 1 ...
 $ em         : int  4 4 4 4 4 4 4 4 4 4 ...
 $ ed         : int  5 5 5 5 5 5 5 5 5 5 ...
 $ ey         : int  2016 2016 2016 2016 2016 2016 2016 2016 2016 ...
 $ early_ship_time : num  12.6 12.6 12.6 12.6 12.6 ...
 $ lm         : int  7 7 7 7 7 7 7 7 7 ...
 $ ld         : int  15 15 15 15 15 15 15 15 15 ...
 $ ly         : int  2016 2016 2016 2016 2016 2016 2016 2016 2016 ...
 $ late_ship_time : num  12.6 12.6 12.6 12.6 12.6 ...
 $ em.l       : int  4 4 4 4 4 4 4 4 4 4 ...
 $ ed.l       : int  12 12 12 12 12 12 12 12 12 ...
 $ ey.l       : int  2016 2016 2016 2016 2016 2016 2016 2016 2016 ...
 $ ext_time   : num  12.6 12.6 12.6 12.6 12.6 ...
 $ calc_pri   : int  6 6 6 6 6 6 6 6 6 ...
 $ cart_time  : num  11.2 11.2 11.2 11.2 11.2 ...
 $ performance_rate : num  0.15 0.15 0.15 0.15 0.15 0.15 0.15 0.15 ...
 $ performance : num  89 89 89 89 89 ...
```

Orders Pick and Pack Performance Analysis

Traveled Time Performance Based on Pick and Pack Stages



Total time Traveled States (TS_t):

$$TS_t = t_{(0,1)} + t_{(1,2)} + t_{(2,3)}$$

$$t_1 =$$

$$\sum_{i=n}^{i=0} tLoc_i$$

$$t_2 =$$

$$\sum_{i=n}^{i=0} tI_i$$

$$t_{(1,2)} = \sum_{i=n}^{i=0} tLoc_i + tI_i$$

$tLoc_i$ = time to travel to a location, where $i = n$ is the last location.

tI_i = time to pick & put an item, where $i = n$ is the last item.

Stage → S | Real Time Data

Current Value for Stages 1, 2, and 3

S1	S2	S3
$t_{(0,1)}$	$t_{(1,2)}$	$t_{(2,3)}$

Overall Value for all Possible Stages

$$\bigcup_{X=3}^{X=0} SX = \frac{\sum_{i=n}^{i=0} t_{(x, x+1)i}}{n}$$

Stage → S | Approximated/Predicted Data

Average Value for Stages 1, 2, and 3 based on prior data

$S1_{avg}$	$S2_{avg}$	$S3_{avg}$
$\sum_{i=n}^{i=0} t_{(0,1)i}/n$	$\sum_{i=n}^{i=0} t_{(1,2)i}/n$	$\sum_{i=n}^{i=0} t_{(2,3)i}/n$

Overall Average Value for all Possible Stages

$$\bigcup_{X=3}^{X=0} SX_{avg} = \frac{\sum_{i=n}^{i=0} t_{(x, x+1)i}}{n}$$

U	S1	S2	S3
S1	1	0	0
S2	0	1	0
S3	0	0	1
TS	1	1	1

$$\therefore TS_t = S1 + S2 + S3$$

Performance Indicator:

$$TS_p = 100\% - \left(\frac{\sqrt{(TS_t + \Delta TS_t)}}{R_{exp}} \right)$$

TS_p = Traveled Stages Performance in %

R_{exp} = Individual Exponential Growth Rate

PIR_{avg} = Overall Exponential Growth Rate

$$\Delta TS_t = S1_{avg} + S2_{avg} + S3_{avg}$$

Performance Indicator Rate (PIR):	Average PIR
R_{exp} $= \frac{\sqrt{(TS_t^2 + \Delta TS_t^2)}}{100}$	$PIR_{avg} = \frac{\sum_{i=n}^{i=0} R_{exp_i}}{n}$

For a 100% Individual Performance Rate

$$R_{exp} = \frac{\sqrt{(100^2 + 100^2)}}{100} = 1.4142 \approx 1.4$$

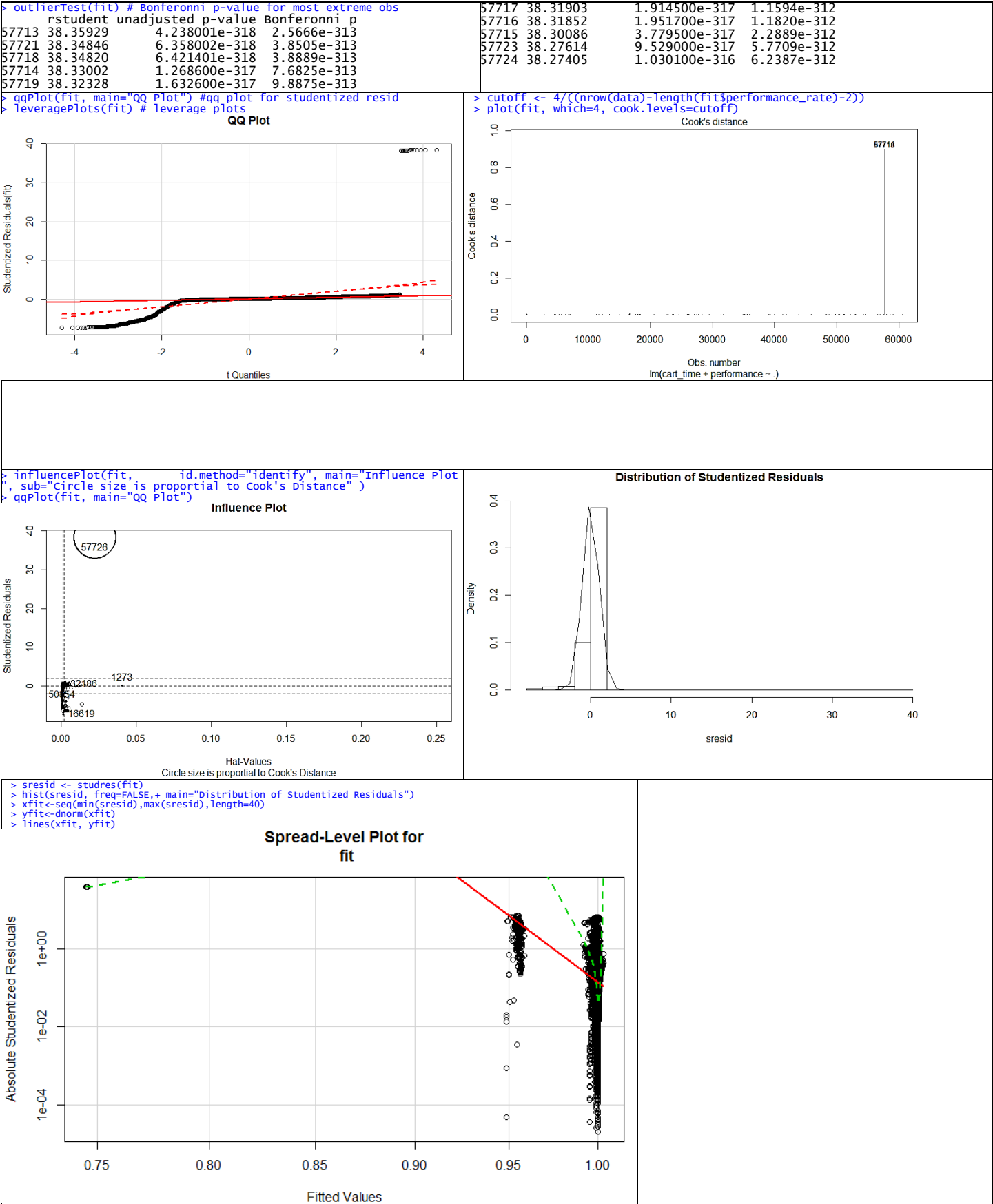
For a 100% Overall Performance Rate

$$PIR_{avg} = \frac{\sum_{i=n}^{i=0} 1.41}{n}$$

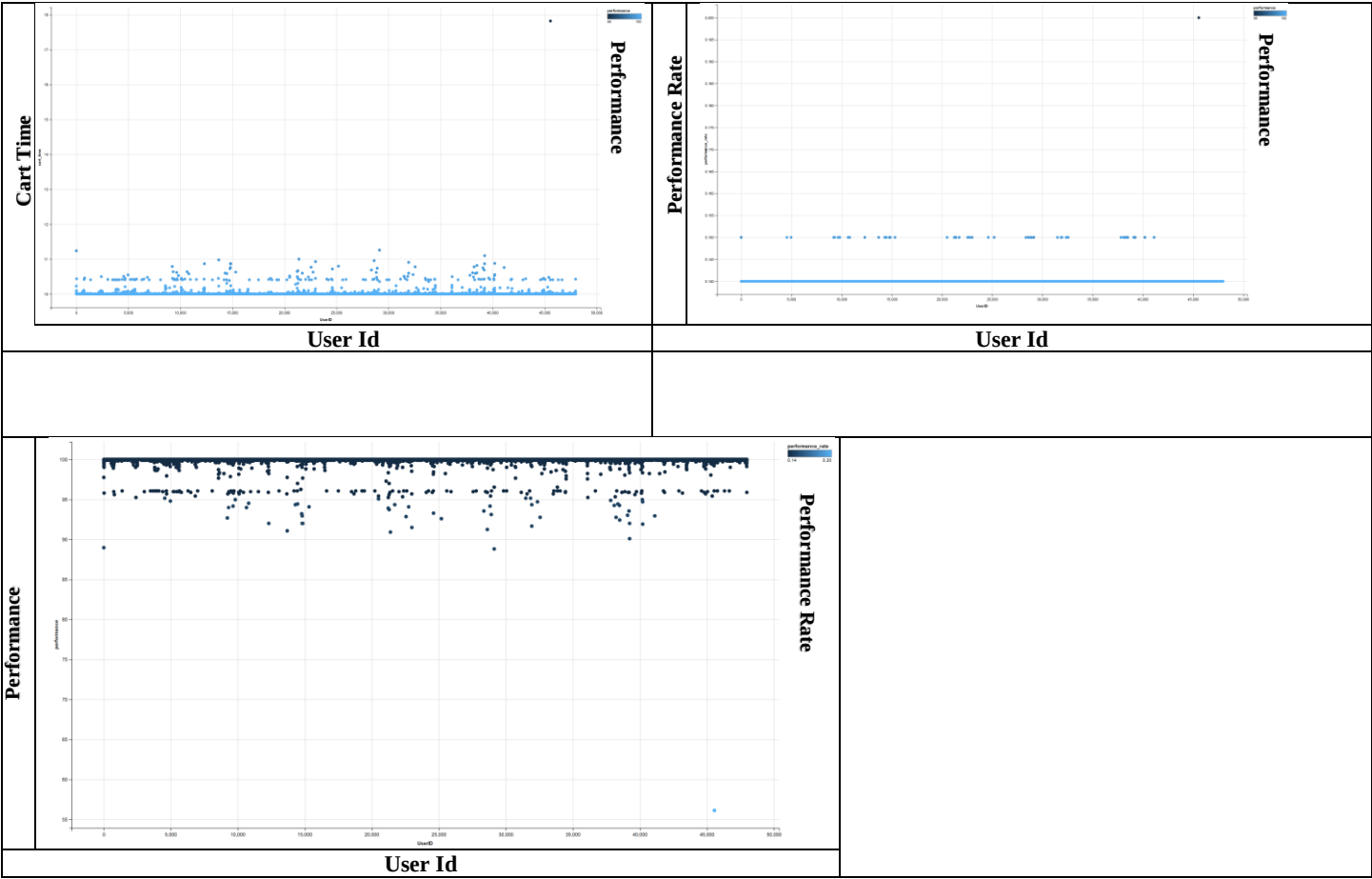
- Handling Invalid Data
 - Empty Values
 - Nominal Data
 - Nominal/Numeric Data

Data Exploration

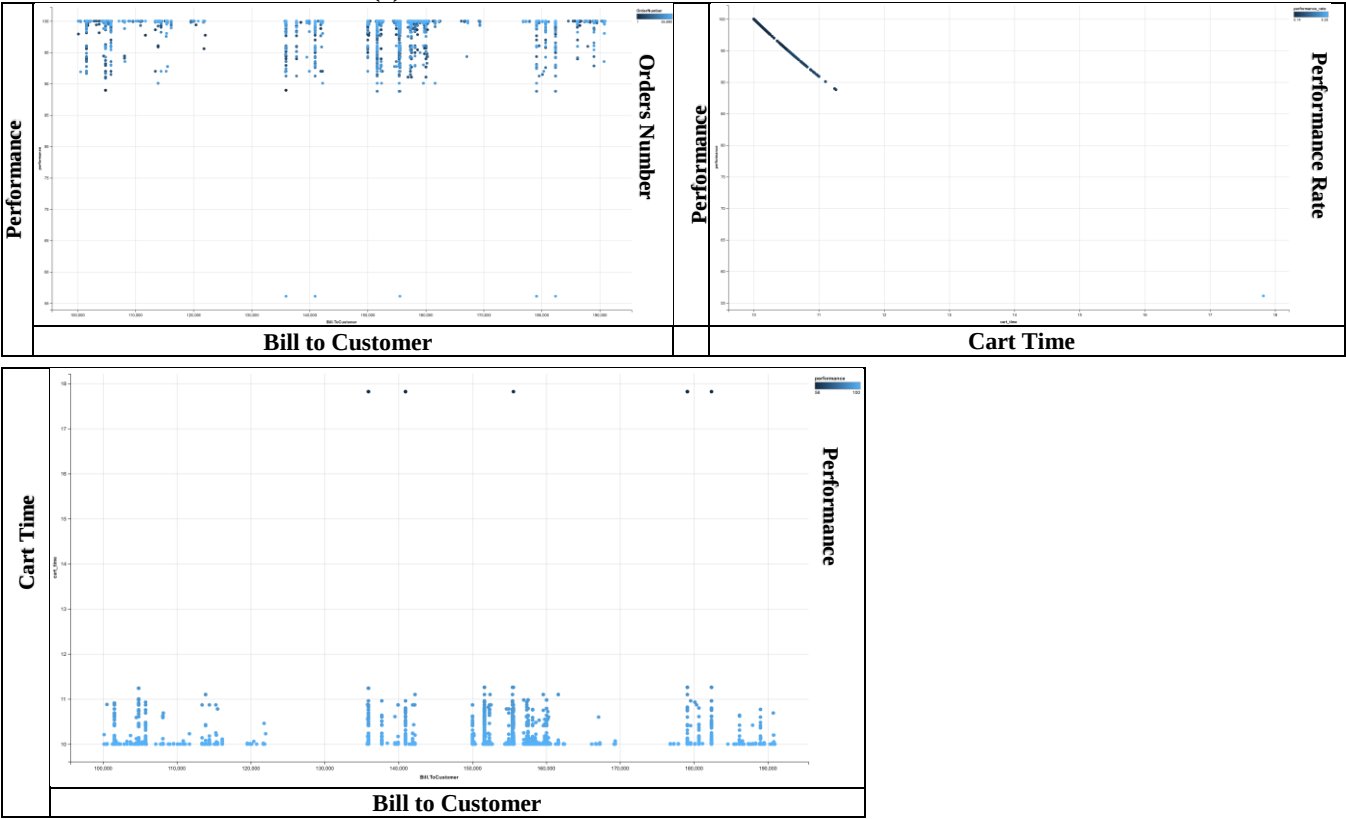
- Observe Data Behavior



- Observe Classes Performance



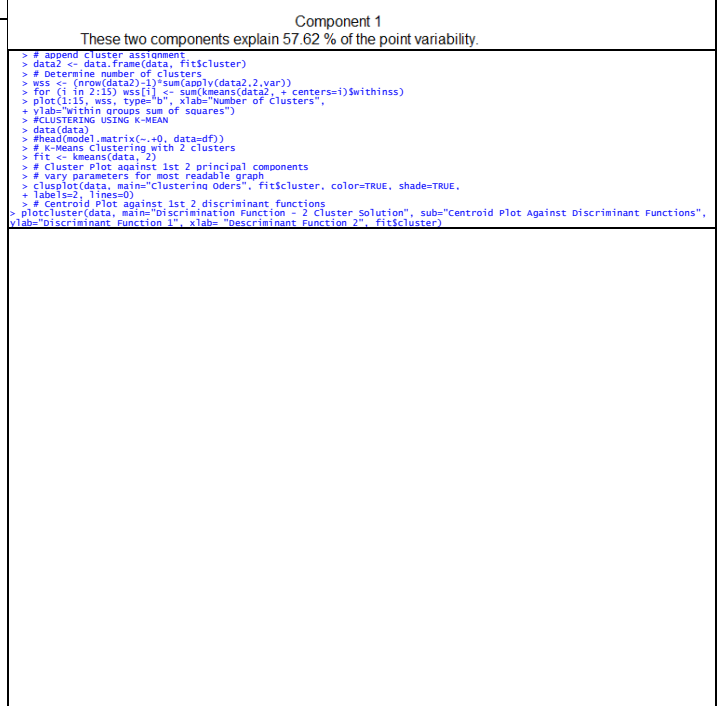
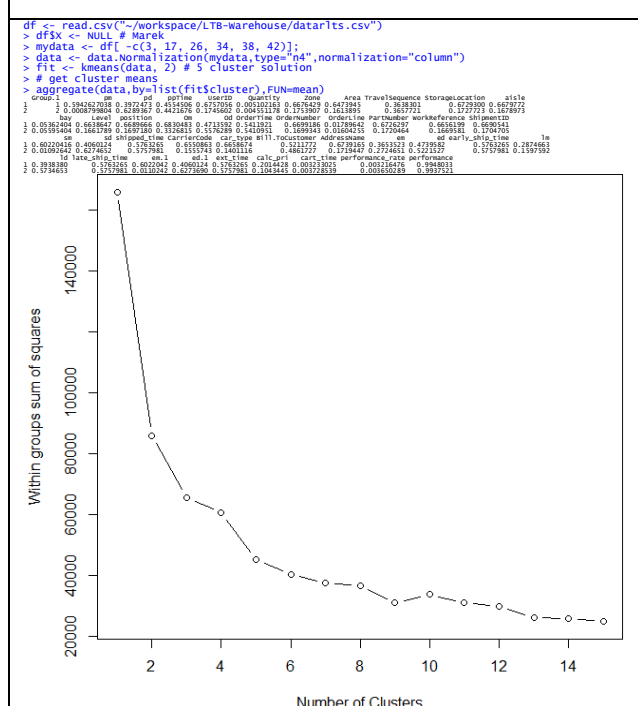
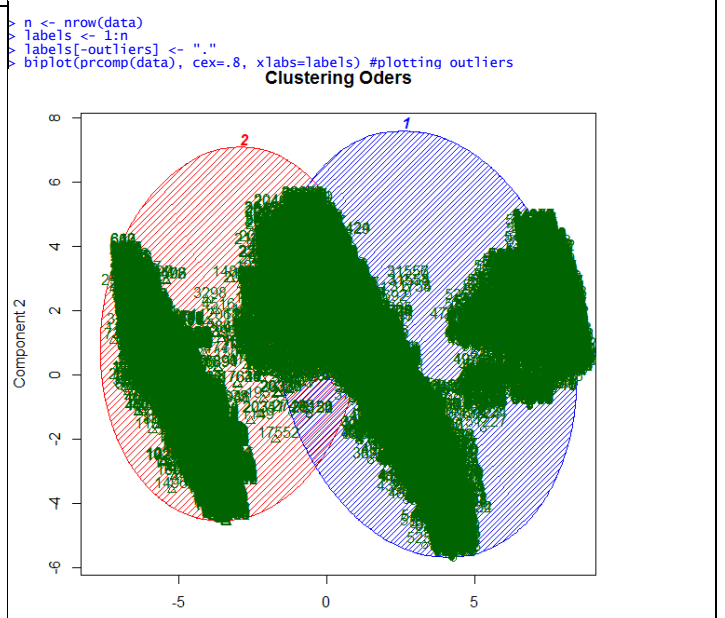
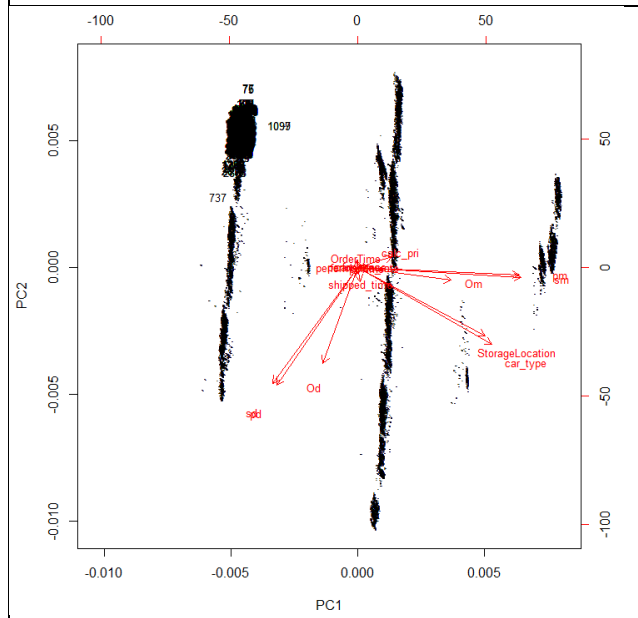
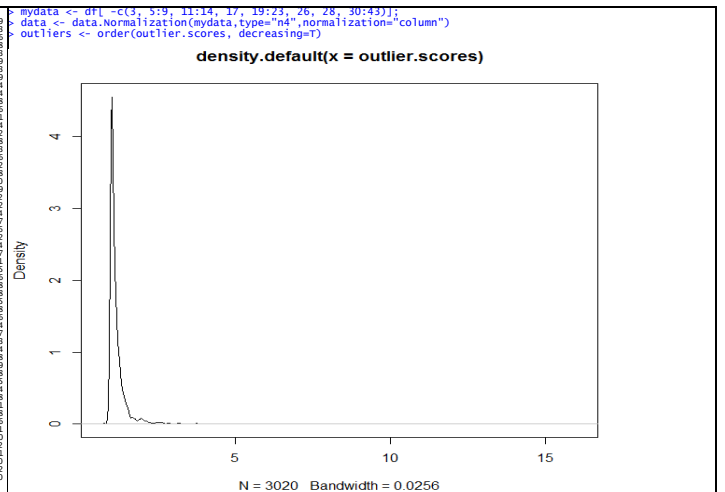
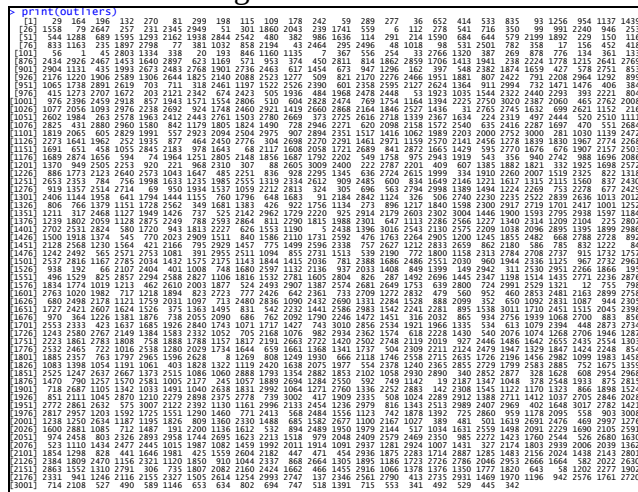
- Observe a Class attribute(s) Performance



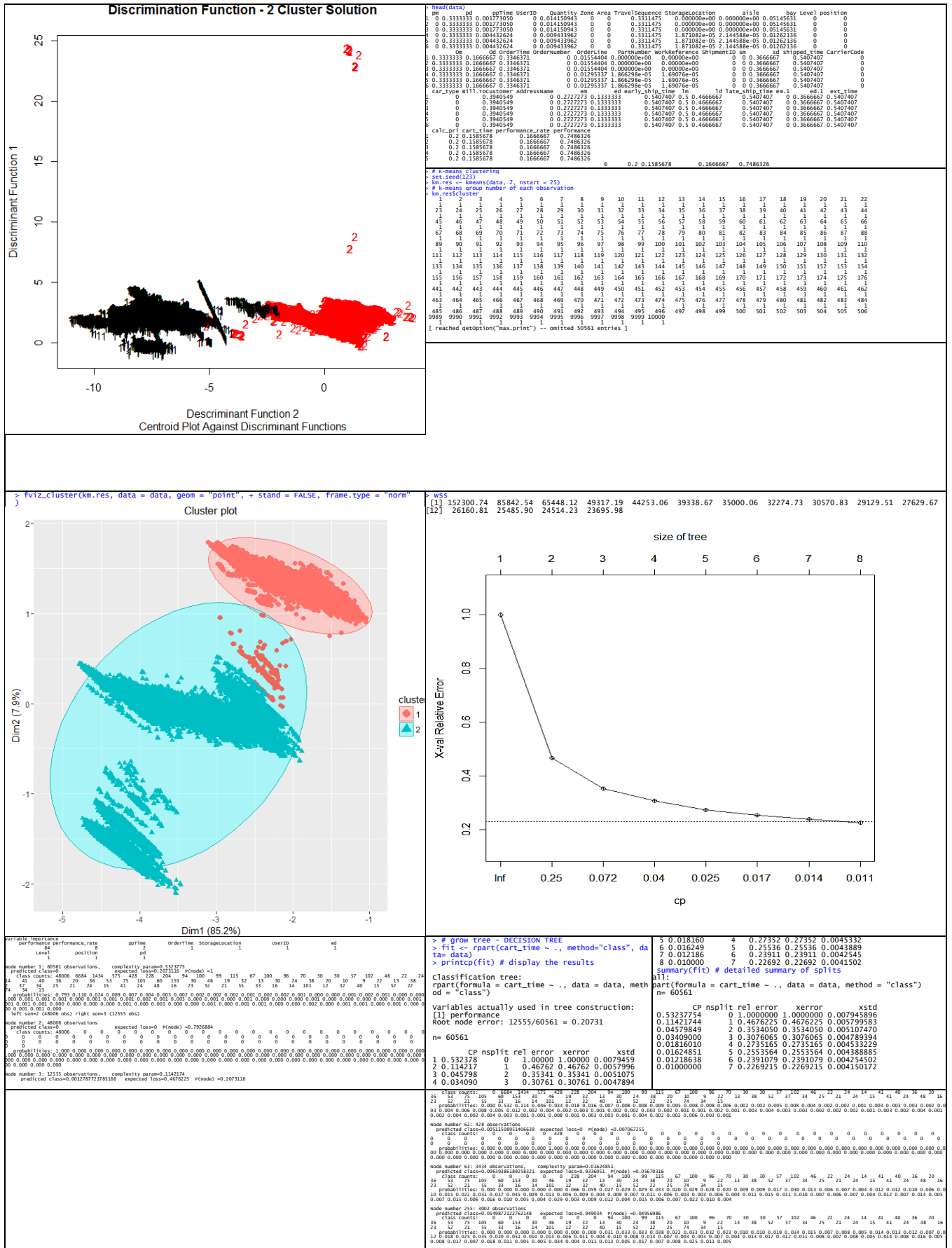
Data Mining

- Clustering Technique

o Data Modeling



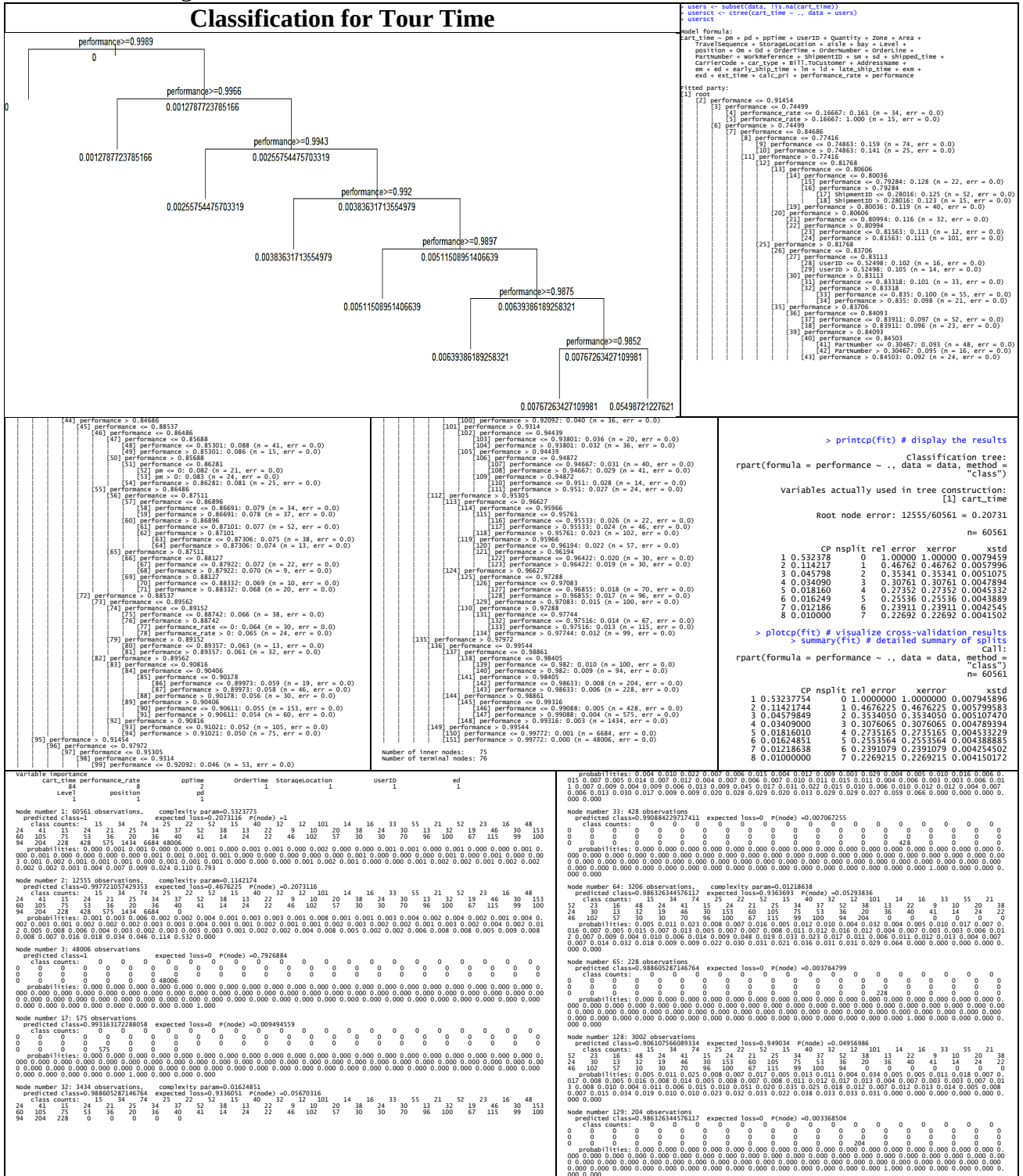
Results



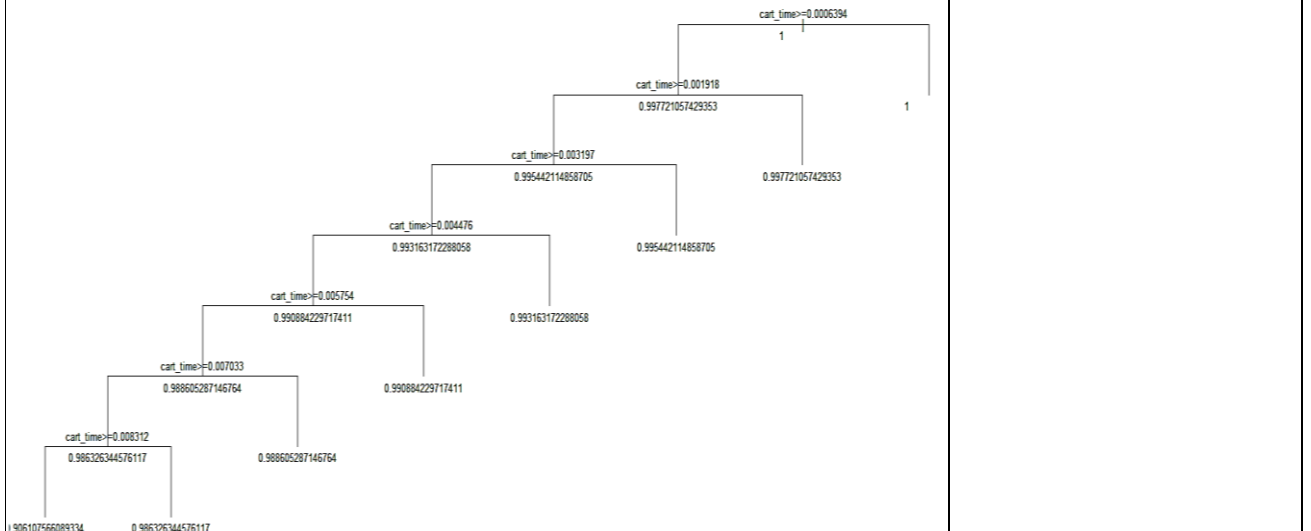
- Validation – Cluster

- Classification Technique

- Data Modeling



Classification Tree for Kyphosis



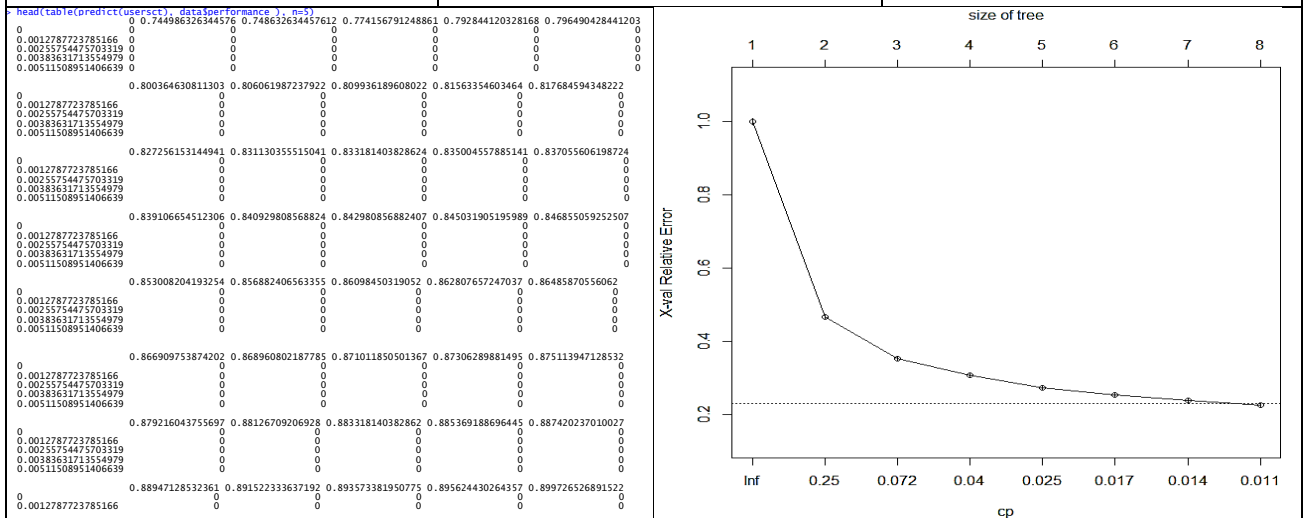
```
1306107566089334 0.986326344576117
usrsct <- ctree(performance ~., data = data)
usrsct

Model Formula:
performance ~ pm + pd + pptime + usrsct + Quantity + Zone + Area +
  TravelSequence + storageLocation + aisle + bay + Level +
  position + On + sd + OrderTime + OrderNumber + OrderTime
PartNumber + workReference + shipmentID + sm + sd + shipped_time +
  em + ed + early_ship_time + ln + ld + late_ship_time + em +
  end + ext_time + cart_time + cart_time + performance_rate

Fitted par:
[1] root
[2]
[3] cart_time <= 0.03964
[4] cart_time <= 0.00895
[5] cart_time <= 0.10028
[6] cart_time > 0.10028
[7] cart_time <= 0.00128
[8] cart_time <= 0.00084
[9] cart_time <= 0.00256: 0.995 (n = 1434, err = 0.0)
[10] cart_time > 0.00256: 0.993 (n = 575, err = 0.0)
[11] cart_time <= 0.00084
[12] cart_time <= 0.00039
[13] cart_time <= 0.00512: 0.991 (n = 428, err = 0.0)
[14] cart_time > 0.00512: 0.989 (n = 228, err = 0.0)
[15] cart_time <= 0.00063
[16] cart_time <= 0.00267: 0.986 (n = 204, err = 0.0)
[17] cart_time > 0.00267: 0.984 (n = 94, err = 0.0)
[18] cart_time <= 0.00895
[19] cart_time <= 0.02046
[20] cart_time <= 0.01407
[21] cart_time <= 0.01151
[22] cart_time <= 0.01023: 0.982 (n = 100, err = 0.0)
[23] cart_time > 0.01023: 0.980 (n = 99, err = 0.0)
[24] cart_time <= 0.01151
[25] cart_time <= 0.01279: 0.977 (n = 115, err = 0.0)
[26] cart_time > 0.01279: 0.975 (n = 67, err = 0.0)
[27] cart_time <= 0.01407
[28] cart_time <= 0.01662
[29] cart_time <= 0.01535: 0.973 (n = 100, err = 0.0)
[30] cart_time > 0.01535: 0.971 (n = 96, err = 0.0)
[31] cart_time <= 0.01662
[32] cart_time <= 0.01029: 0.969 (n = 70, err = 0.0)
[33] cart_time > 0.01029: 0.967 (n = 70, err = 0.0)
[34] cart_time <= 0.01018: 0.966 (n = 30, err = 0.0)
[35] cart_time > 0.01018: 0.964 (n = 30, err = 0.0)
[36] cart_time <= 0.00946
[37] cart_time <= 0.00813
[38] cart_time <= 0.0243
[39] cart_time <= 0.02274: 0.962 (n = 57, err = 0.0)
[40] cart_time > 0.02274
[41] cart_time <= 0.02302: 0.960 (n = 102, err = 0.0)
[42] cart_time > 0.02302: 0.958 (n = 46, err = 0.0)
[43] cart_time > 0.0243

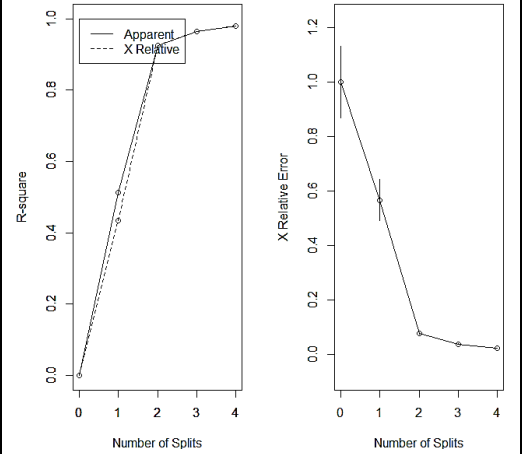
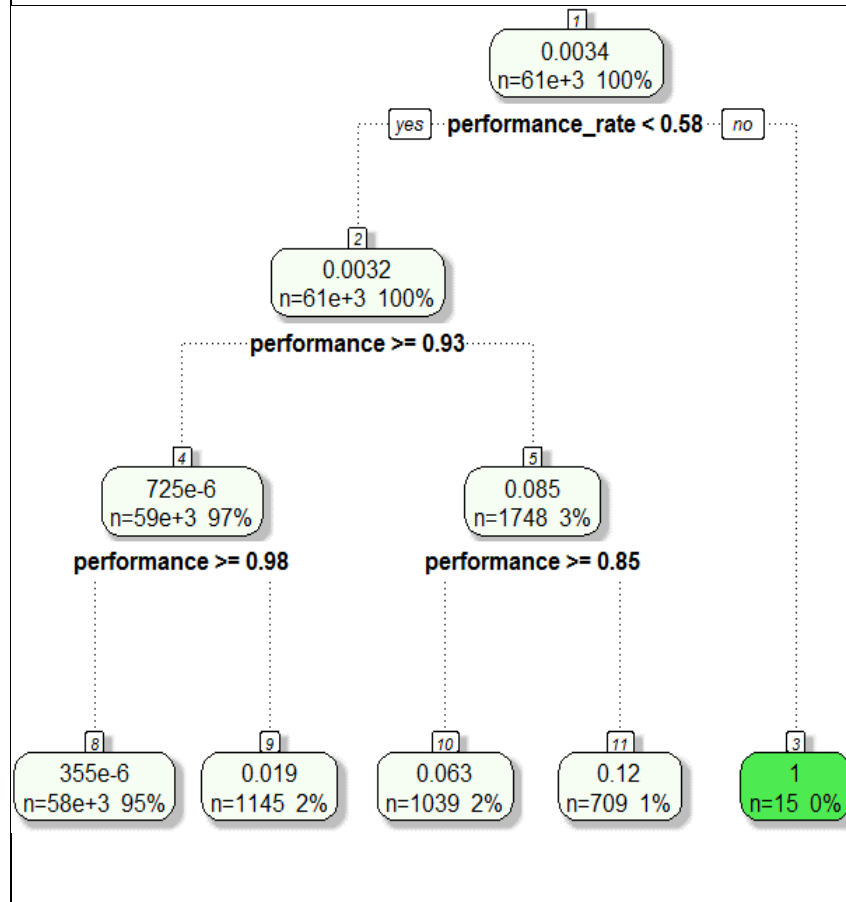
[44] cart_time <= 0.02558: 0.955 (n = 24, err = 0.0)
[45] cart_time > 0.02558
[46] cart_time <= 0.02685: 0.953 (n = 24, err = 0.0)
[47] cart_time > 0.02685: 0.951 (n = 14, err = 0.0)
[48] cart_time <= 0.02613
[49] cart_time <= 0.03197
[50] cart_time <= 0.03069
[51] cart_time <= 0.02941: 0.949 (n = 41, err = 0.0)
[52] cart_time > 0.02941: 0.947 (n = 40, err = 0.0)
[53] cart_time <= 0.03069: 0.944 (n = 36, err = 0.0)
[54] cart_time > 0.03197
[55] cart_time <= 0.03581: 0.938 (n = 20, err = 0.0)
[56] cart_time > 0.03581: 0.931 (n = 36, err = 0.0)
[57] cart_time > 0.03964
[58] cart_time <= 0.03857
[59] cart_time <= 0.08568
[60] cart_time <= 0.08394
[61] cart_time <= 0.05627
[62] cart_time <= 0.04887
[63] ld <= 0.16667: 0.915 (n = 75, err = 0.0)
[64] ld > 0.16667: 0.921 (n = 53, err = 0.0)
[65] cart_time > 0.04887
[66] cart_time <= 0.05371
[67] cart_time <= 0.05243: 0.910 (n = 105, err = 0.0)
[68] cart_time > 0.05243: 0.908 (n = 60, err = 0.0)
[69] cart_time <= 0.05371
[70] cart_time <= 0.05499: 0.906 (n = 153, err = 0.0)
[71] cart_time > 0.05499: 0.904 (n = 30, err = 0.0)
[72] cart_time <= 0.05627
[73] cart_time <= 0.05882
[74] cart_time <= 0.05754: 0.902 (n = 46, err = 0.0)
[75] cart_time > 0.05754: 0.900 (n = 19, err = 0.0)
[76] cart_time > 0.05882
[77] cart_time <= 0.06138: 0.896 (n = 32, err = 0.0)
[78] cart_time > 0.06138
[79] cart_time <= 0.06266: 0.894 (n = 13, err = 0.0)
[80] cart_time > 0.06266: 0.892 (n = 30, err = 0.0)
[81] cart_time > 0.06394
[82] cart_time <= 0.07161
[83] cart_time <= 0.06777
[84] cart_time <= 0.06522: 0.889 (n = 24, err = 0.0)
[85] cart_time > 0.06522
[86] cart_time <= 0.06665: 0.887 (n = 38, err = 0.0)
[87] cart_time > 0.06665: 0.885 (n = 20, err = 0.0)
[88] cart_time <= 0.06777
[89] cart_time <= 0.07033: 0.882 (n = 19, err = 0.0)
[90] cart_time > 0.07033: 0.879 (n = 22, err = 0.0)
[91] cart_time > 0.07161
[92] cart_time <= 0.07028
[93] cart_time <= 0.07673
[94] cart_time <= 0.07545
[95] cart_time <= 0.07417: 0.875 (n = 13, err = 0.0)
[96] cart_time > 0.07417: 0.873 (n = 18, err = 0.0)
[97] cart_time > 0.07545: 0.871 (n = 52, err = 0.0)
[98] cart_time > 0.07673

[99] cart_time <= 0.07801: 0.869 (n = 37, err = 0.0)
[100] cart_time > 0.07801: 0.867 (n = 34, err = 0.0)
[101] cart_time <= 0.07938
[102] cart_time <= 0.08312
[103] cart_time <= 0.08056: 0.865 (n = 25, err = 0.0)
[104] cart_time > 0.08056
[105] sm <= 0: 0.863 (n = 21, err = 0.0)
[106] sm > 0: 0.861 (n = 24, err = 0.0)
[107] cart_time <= 0.08312: 0.857 (n = 15, err = 0.0)
[108] cart_time > 0.08312
[109] cart_time <= 0.11893
[110] cart_time <= 0.1023
[111] cart_time <= 0.09463
[112] cart_time <= 0.08824: 0.853 (n = 41, err = 0.0)
[113] cart_time > 0.08824
[114] cart_time <= 0.09207: 0.847 (n = 24, err = 0.0)
[115] cart_time > 0.09207
[116] PartNumber <= 0.30467: 0.845 (n = 48, err = 0.0)
[117] PartNumber > 0.30467: 0.843 (n = 16, err = 0.0)
[118] cart_time > 0.09463
[119] cart_time <= 0.09847
[120] cart_time <= 0.09719
[121] cart_time <= 0.09591: 0.841 (n = 23, err = 0.0)
[122] cart_time > 0.09591: 0.839 (n = 52, err = 0.0)
[123] cart_time <= 0.09719: 0.837 (n = 21, err = 0.0)
[124] cart_time > 0.09847
[125] cart_time <= 0.09974: 0.835 (n = 55, err = 0.0)
[126] cart_time > 0.09974
[127] cart_time <= 0.10102: 0.833 (n = 33, err = 0.0)
[128] cart_time > 0.10102: 0.831 (n = 16, err = 0.0)
[129] cart_time <= 0.1023
[130] cart_time <= 0.11253
[131] cart_time <= 0.10486: 0.827 (n = 14, err = 0.0)
[132] cart_time > 0.10486
[133] cart_time <= 0.11125: 0.818 (n = 101, err = 0.0)
[134] cart_time > 0.11125: 0.816 (n = 32, err = 0.0)
[135] cart_time <= 0.11253
[136] cart_time <= 0.11637: 0.810 (n = 32, err = 0.0)
[137] cart_time > 0.11637: 0.806 (n = 40, err = 0.0)
[138] cart_time > 0.11893
[139] cart_time <= 0.14066
[140] cart_time <= 0.12788
[141] cart_time <= 0.12532
[142] ShipmentID <= 0.28016: 0.796 (n = 52, err = 0.0)
[143] ShipmentID > 0.28016: 0.800 (n = 15, err = 0.0)
[144] cart_time <= 0.12532: 0.793 (n = 22, err = 0.0)
[145] cart_time > 0.12788: 0.774 (n = 21, err = 0.0)
[146] cart_time > 0.14066: 0.749 (n = 74, err = 0.0)
[147] cart_time <= 0.15857
[148] sm <= 0.5: 0.745 (n = 34, err = 0.0)
[149] sm > 0.5: 0.900 (n = 15, err = 0.0)
```



○ Results

Tour Time Classification Results



```

> fancyRpartPlot(fit)
> # create additional plots
> par(mfrow=c(1,2)) # two plots on one page
> rsq.rpart(fit) # visualize cross-validation results

```

Regression tree:
rpart(formula = cart_time ~ ., data = data, method = "anova")

Variables actually used in tree construction:
[1] performance performance_rate

Root node error: 29.11/60561 = 0.00048067

n= 60561

CP	nsplit	rel error	xerror	xstd
1	0.511915	0	1.000000	1.000038
2	0.412447	1	0.488085	0.566316
3	0.040476	2	0.075638	0.076034
4	0.013948	3	0.035162	0.036245
5	0.010000	4	0.021214	0.021463

```

> fit <- rpart(performance ~ .,
+ method="anova", data=data)
> printcp(fit) # display the results

```

Regression tree:
rpart(formula = performance ~ ., data = data, method = "anova")

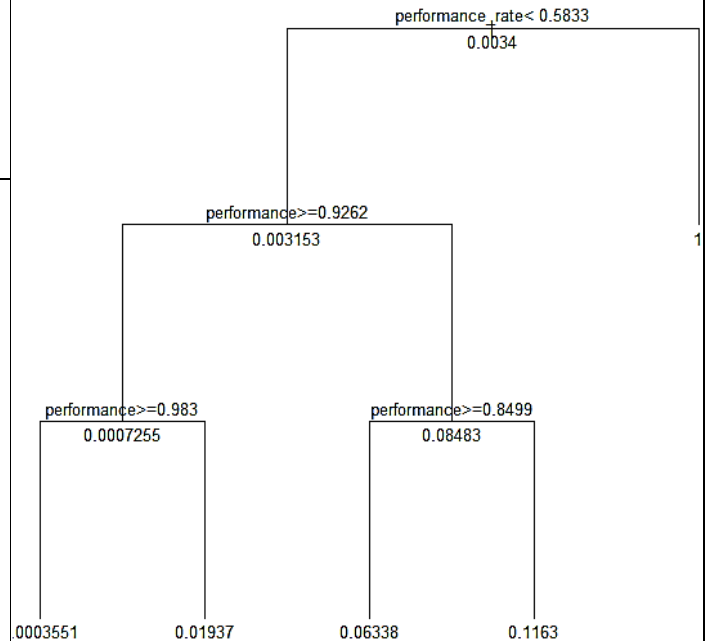
Variables actually used in tree construction:
[1] cart_time performance_rate

Root node error: 53.42/60561 = 0.00088208

n= 60561

CP	nsplit	rel error	xerror	xstd
1	0.689563	0	1.000000	1.000020
2	0.205589	1	0.310437	0.310766
3	0.053046	2	0.104848	0.104904
4	0.032391	3	0.051802	0.051833
5	0.010342	4	0.028511	0.028560
6	0.010000	5	0.018169	0.020751

Regression Tree for Users



```

> plot(usrsct2)
> fit <- rpart(cart_time ~ .,
+ method="anova", data=data)
> printcp(fit) # display the results

```

Regression tree:
rpart(formula = cart_time ~ ., data = data, method = "anova")

Variables actually used in tree construction:
[1] performance performance_rate

Root node error: 29.11/60561 = 0.00048067

n= 60561

CP	nsplit	rel error	xerror	xstd
1	0.511915	0	1.000000	1.000038
2	0.412447	1	0.488085	0.566316
3	0.040476	2	0.075638	0.076034
4	0.013948	3	0.035162	0.036245
5	0.010000	4	0.021214	0.021463

Variable importance

performance performance_rate

Area 52 43

1 1

Node number 1: 60561 observations,

mean=0.003400389, MSE=0.0004806727

Node number 2: 60546 observations,

mean=0.003153486, MSE=0.0002346673

Node number 3: 15 observations

mean=1, MSE=0

Node number 4: 58798 observations,

mean=0.000725467, MSE=0.059894e-06

Node number 5: 1748 observations,

mean=0.08482548, MSE=0.0009548757

Node number 8: 57653 observations

mean=0.000355132, MSE=1.067799e-06

Node number 9: 1145 observations

mean=0.01937256, MSE=5.685768e-05

Node number 10: 1039 observations

mean=0.06337862, MSE=0.0001490049

Node number 11: 709 observations

mean=0.1162547, MSE=0.0004739838

Node number 11: 709 observations

mean=0.1162547, MSE=0.0004739838

Node number 11: 709 observations

mean=0.1162547, MSE=0.0004739838

Node number 11: 709 observations

mean=0.1162547, MSE=0.0004739838

Node number 11: 709 observations

mean=0.1162547, MSE=0.0004739838

Node number 11: 709 observations

mean=0.1162547, MSE=0.0004739838

Node number 11: 709 observations

mean=0.1162547, MSE=0.0004739838

Node number 11: 709 observations

mean=0.1162547, MSE=0.0004739838

Node number 11: 709 observations

mean=0.1162547, MSE=0.0004739838

Node number 11: 709 observations

mean=0.1162547, MSE=0.0004739838

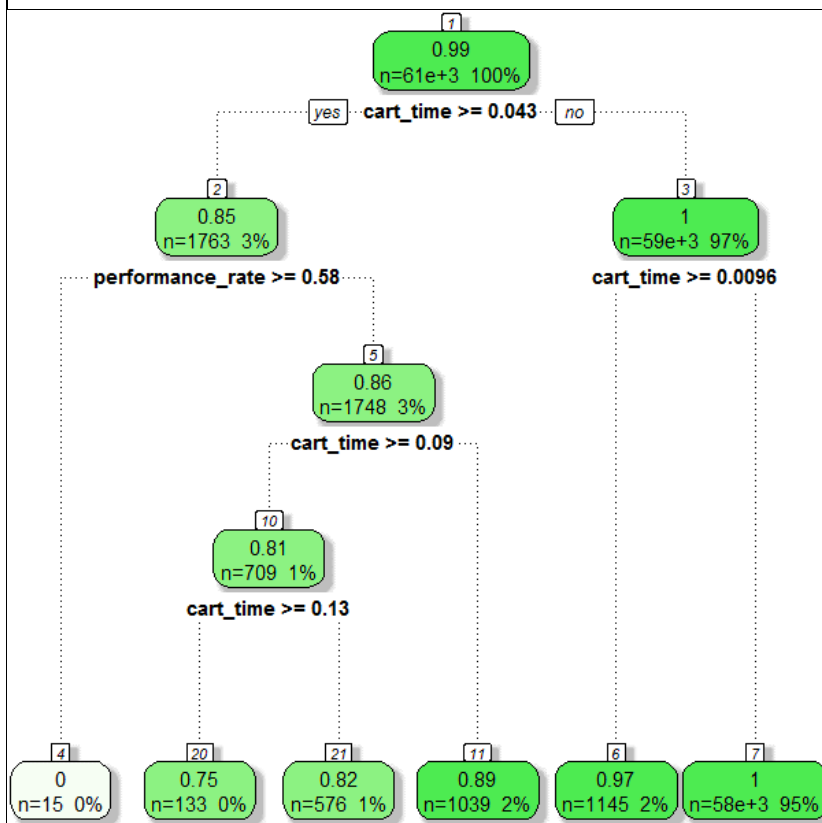
Node number 11: 709 observations

mean=0.1162547, MSE=0.0004739838

Node number 11: 709 observations

mean=0.1162547, MSE=0.0004739838

Performance Classification Results



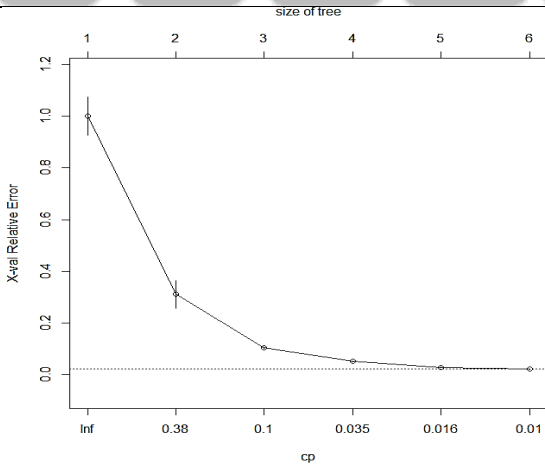
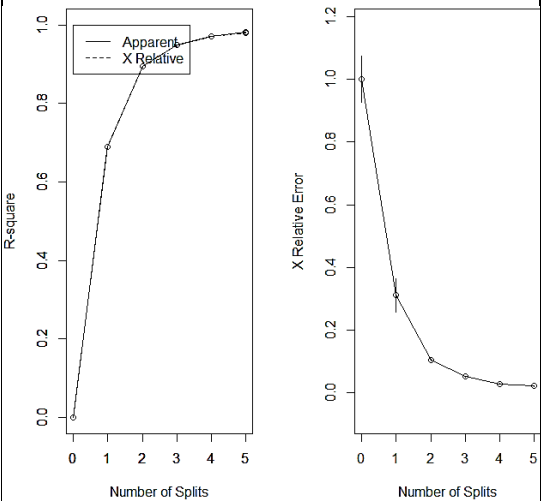
```

> fancyRpartPlot(fit)
> # create additional plots
> par(mfrow=(1,2)) # two plots on one page
> rsq.rpart(fit) # visualize cross-validation results

Regression tree:
rpart(formula = performance ~ ., data = data, method = "anova")
Variables actually used in tree construction:
[1] cart_time performance_rate

Root node error: 53.42/60561 = 0.00088208
n= 60561

```



```

> plotcp(fit) # visualize cross-validation results
> plot(fit, uniform=TRUE, + main="Regression Tree for users ")
> text(fit, use.n=FALSE, all=TRUE, cex=.8)
> summary(fit) # detailed summary of splits

Call:
rpart(formula = performance ~ ., data = data, method = "anova")
n= 60561

   CP nsplit rel error   xerror   xstd
1 0.68956317 0 1.0000000 1.00002035 0.07426802
2 0.20558883 1 0.31043683 0.31076596 0.0527582867
3 0.05304591 2 0.10484799 0.10490363 0.0030623042
4 0.02329100 3 0.05180209 0.05183297 0.0013348304
5 0.01034216 4 0.02851108 0.02856041 0.0009114243
6 0.01000000 5 0.01816892 0.02075087 0.0005790029

Variable importance
Area cart_time performance_rate aisle StorageLocation UserID
2 50 35 3 3 3
Zone
2

```

Node number 1: 60561 observations,
mean=0.9944483, MSE=0.0008820821

Node number 2: 1763 observations,
mean=0.8520198, MSE=0.008480236

Node number 3: 58798 observations,
mean=0.9987188, MSE=2.776981e-05

Node number 4: 15 observations
mean=0, MSE=0

Node number 5: 1748 observations,
mean=0.8593312, MSE=0.002270107

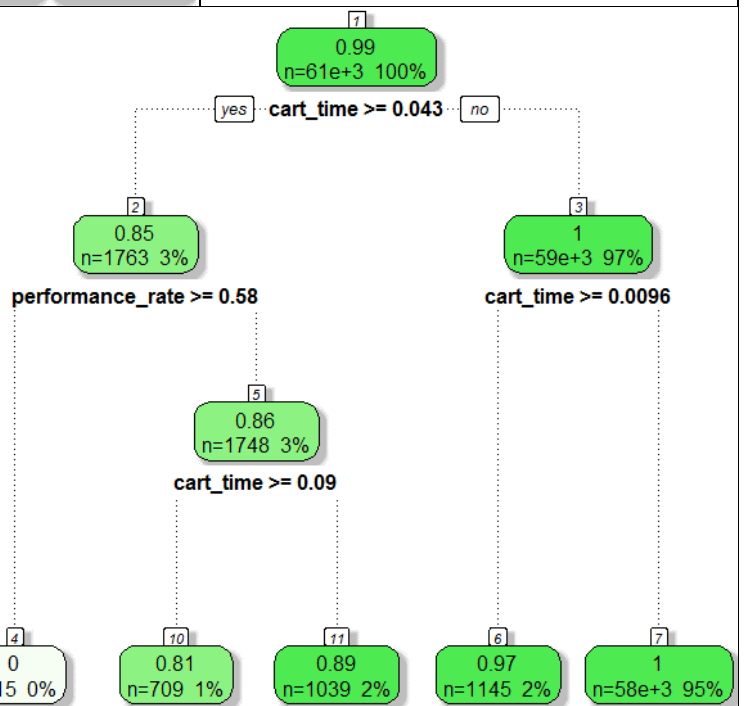
Node number 6: 1145 observations

complexity param=0.6895632

complexity param=0.2055888

complexity param=0.023291

complexity param=0.05304591



mean=0.9660772, MSE=0.0001686366

Node number 7: 57653 observations
mean=0.9993671, MSE=3.391328e-06

Node number 10: 709 observations,
mean=0.8105906, MSE=0.001032947

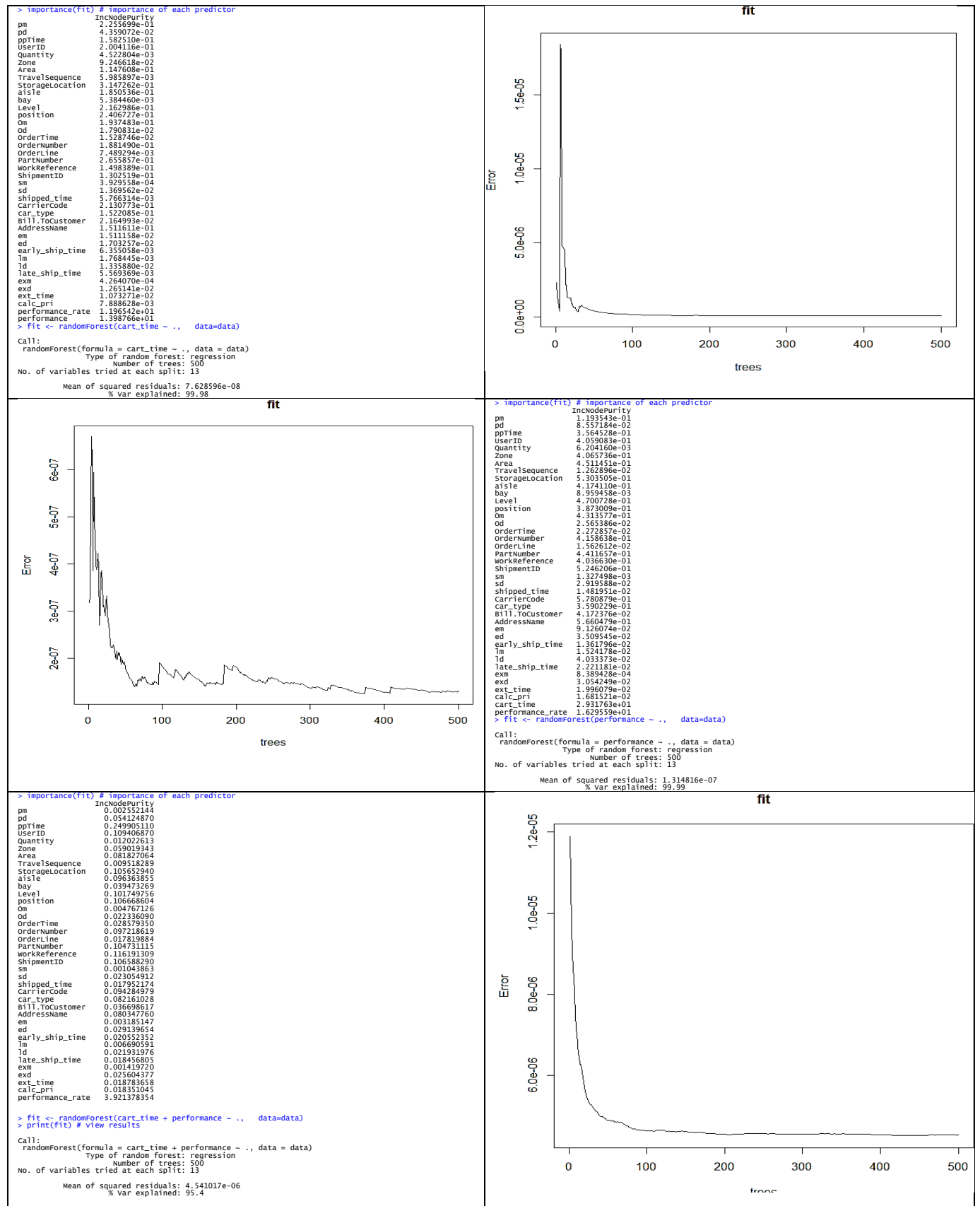
Node number 11: 1039 observations
mean=0.8925912, MSE=0.0003869945

Node number 20: 133 observations
mean=0.7524983, MSE=0.0001109149

Node number 21: 576 observations
mean=0.8240043, MSE=0.0002866868

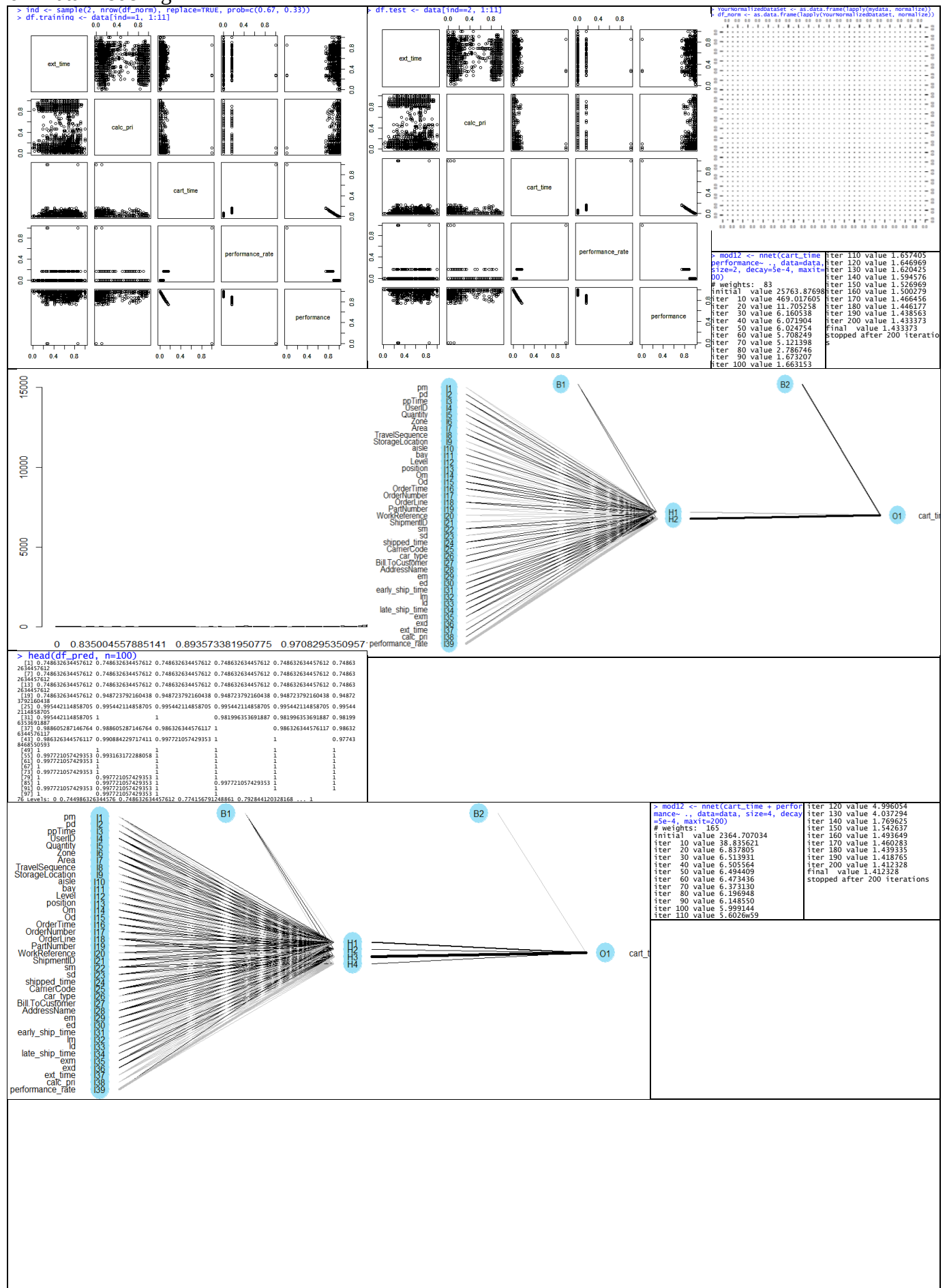
complexity param=0.01034216

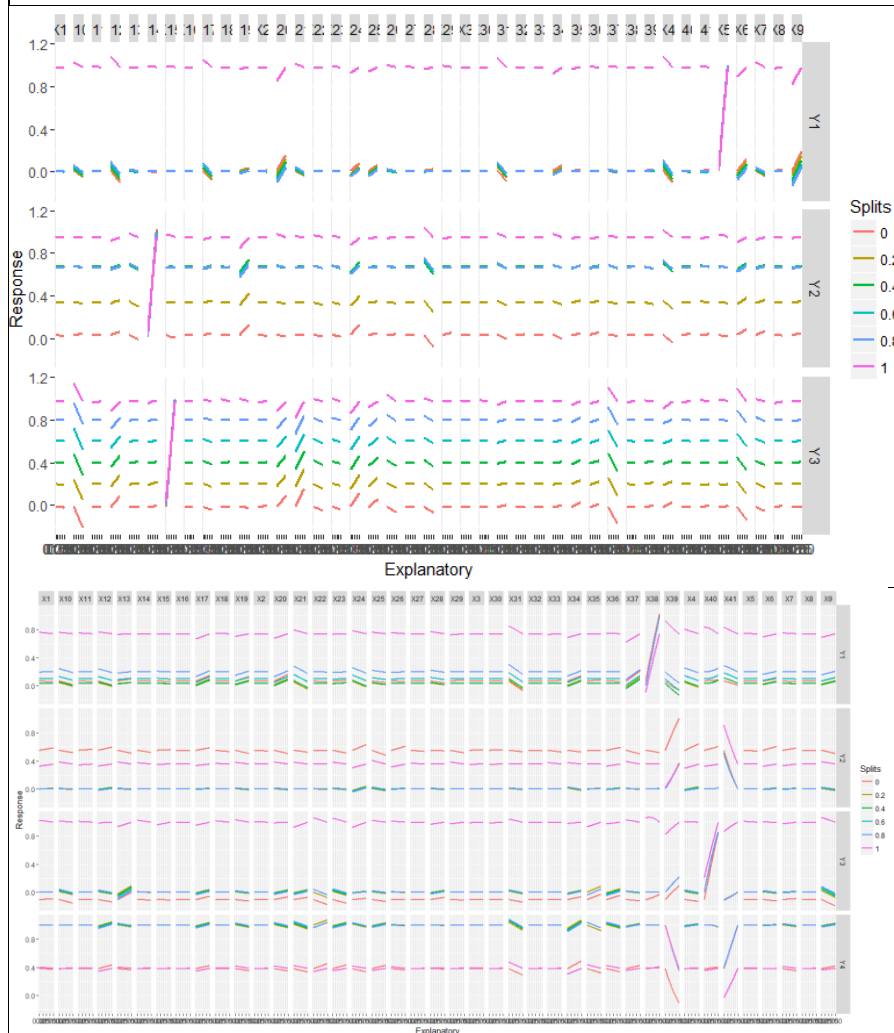
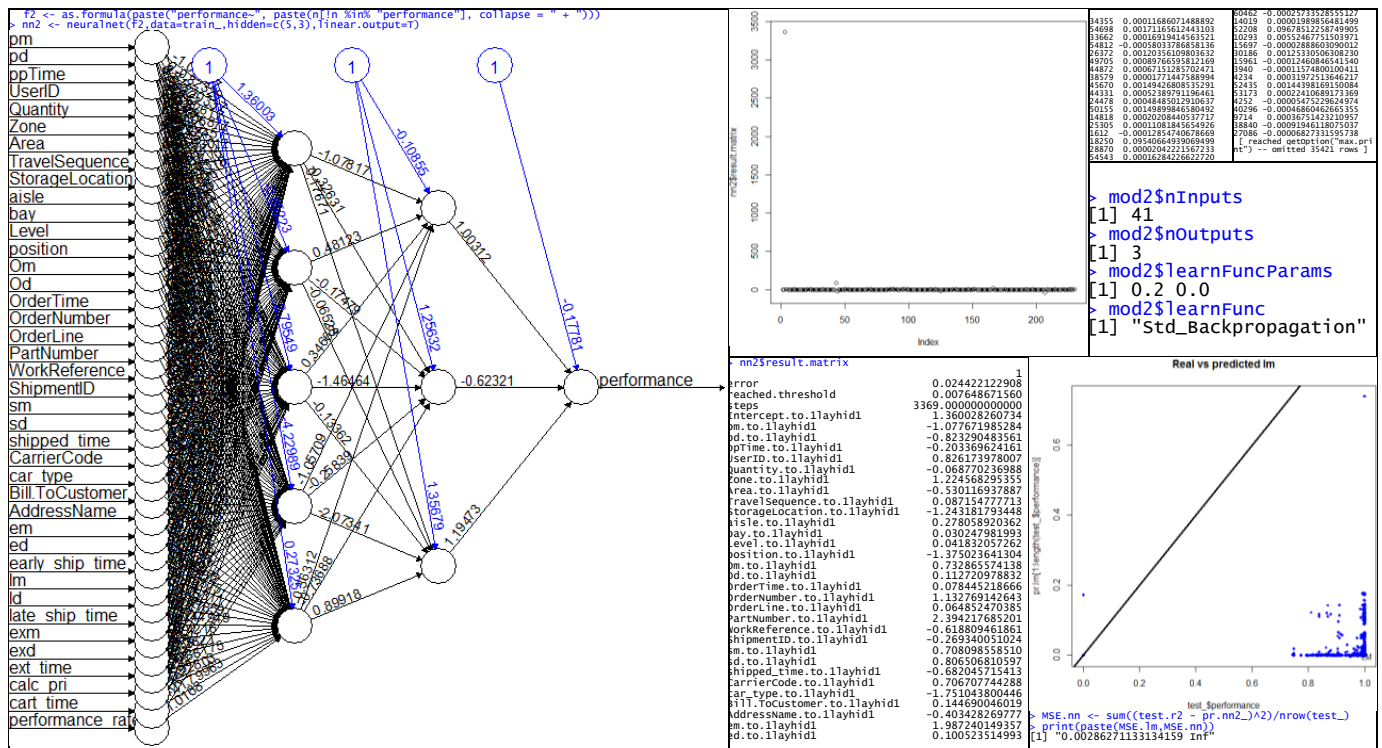
Validation



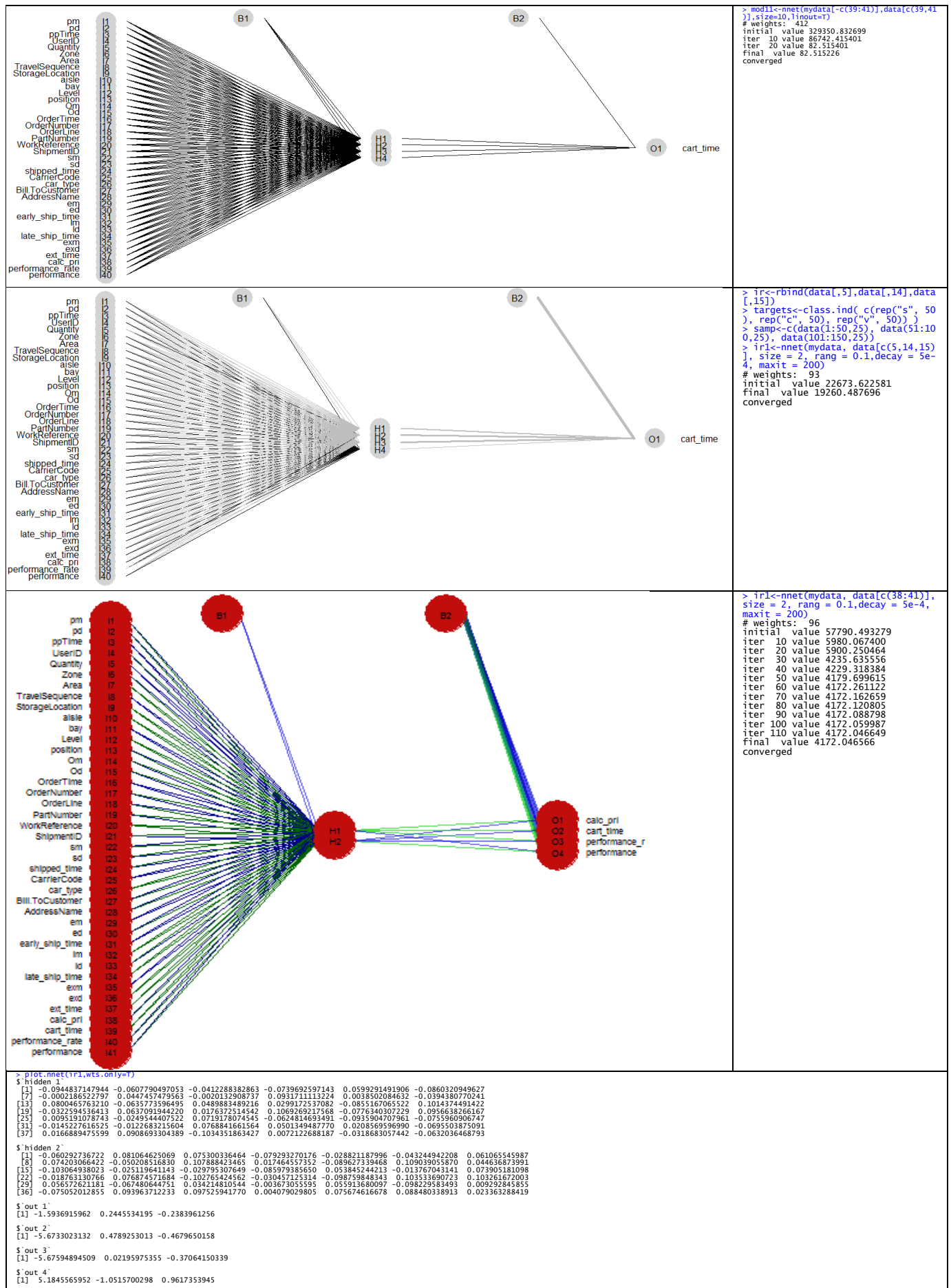
- Advanced Techniques

○ Data Modeling





Validation



SYSTEM IMPLEMENTATION

Introduction

- Motivation
- The Need – based on data analysis
- The Advantages

System Specifications

A Neural Network (NN) can be thought of as a weighted directed acyclic graph. The nodes of the graph are arranged in layers and are connected by weighted edges to nodes in the next layer. The first layer is called the input layer. The last layer is called the output layer. In the case of a regression model, the output layer contains a single node. The remaining layers are called hidden layers. To compute the output of the network on a given input example, a value is calculated for each node in hidden layers and output layer. For each node, the value is set by calculating the weighted sum of the values of nodes in previous layer and applying an activation function to that weighted sum.

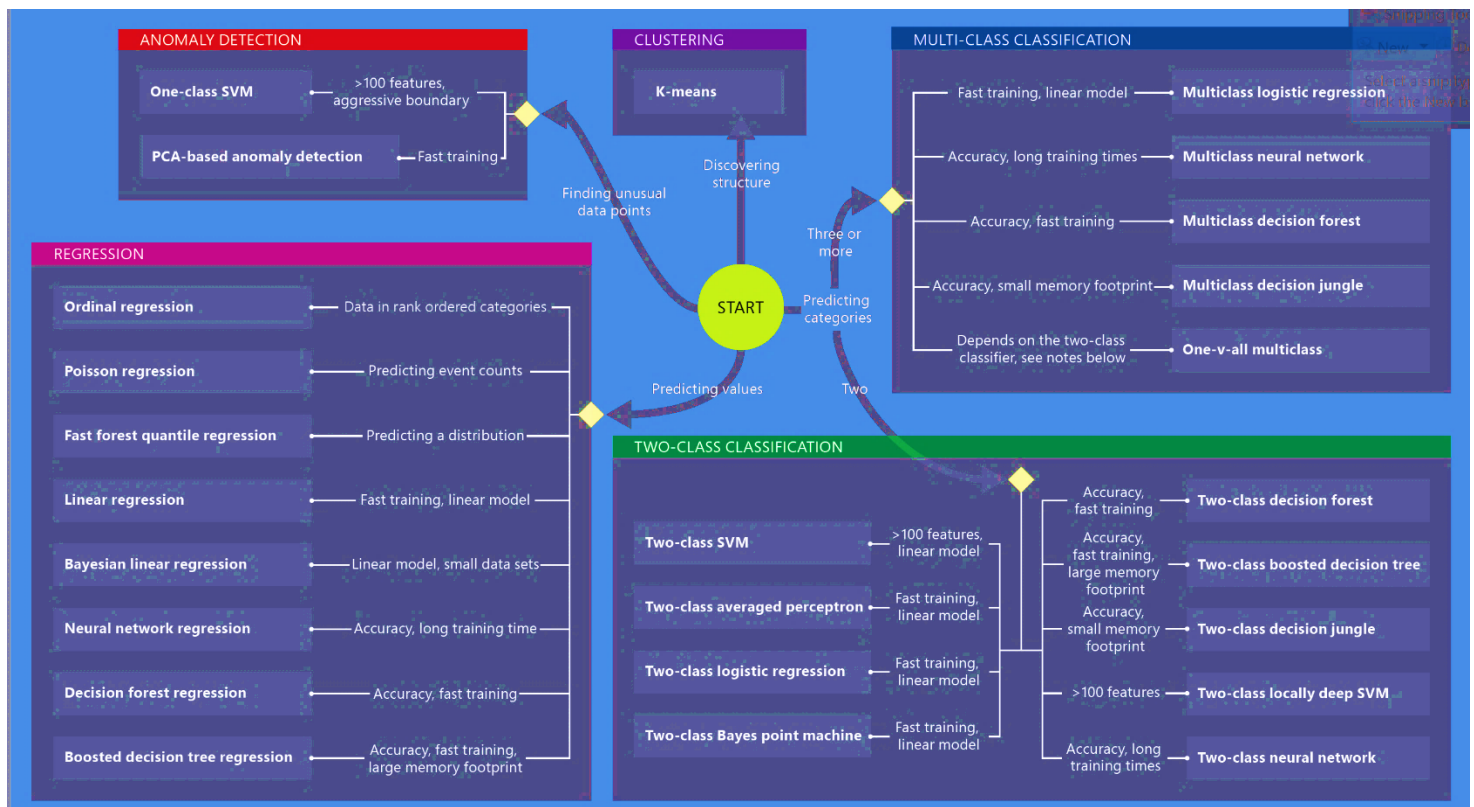
A NN model is defined by the structure of its graph, which includes these attributes:

- Number of hidden layers
- Connections between the layers
- Weights on the graph edges
- Number of nodes in each hidden layer
- Choice of activation functions

The structure of the graph and the activation function are determined by the user. The weights on the edges are learned when training the neural network on the input data.

Microsoft (MS) provide information about the typical workflow for machine learning, which has many phases include the followings:

- Identifying a problem to solve and a metric for measuring results
- Finding, cleaning, and preparing appropriate data
- Identifying the best features and engineering new features
- Building, evaluating, and tuning models
- Using models to generate predictions, recommendations, and other results



MS also provides modules for the final phases of machine learning, in which you apply an algorithm to data to train a model, generate scores, and then evaluate the accuracy and usefulness of the model.

- Initialize Model: Choose from a variety of customizable machine learning algorithms, including *clustering*, *regression*, *classification*, and *anomaly detection* models.
- Train: Provide the configured model with data to learn from patterns and create statistics to use for predictions.
- Score: Create predictions using the trained models.
- Evaluate: Measure the accuracy of a trained model or compare multiple models.

- Platform

MS platform “Visual Studio(VS)” provides a variety of algorithms to implement a NN. However, the “Multiclass NN” approach would fit best because the tasks needed in the LTB-Warehouse project. The *Multiclass NN* module can create a NN model that can be used to predict a target that has multiple values. Classification using neural networks is a *supervised learning* method, and therefore requires a tagged dataset that includes a label column. The model can be trained by providing the model and the tagged dataset as an input to *Train Model* or to *Tune Model Hyperparameters(TMh)*. The trained model can then be used to predict values for the new input examples.

Understanding Neural Network in MS Visual Studio

A NN is a set of interconnected layers, in which the inputs lead to outputs by a series of weighted edges and nodes. The weights on the edges are learned when training the NN on the input data. The direction of the graph proceeds from the inputs through the hidden layer, with all nodes of the graph connected by the weighted edges to nodes in the next layer. Most predictive tasks can be accomplished with only one or a few hidden layers, although recent research has shown that deep neural networks (DNN) can be very effective for complex tasks, in which a successive layer’s model increases the levels of semantic depth. To compute the output of the network for a given input, a value is calculated for each node in the hidden layers and output layer. For each node, the value is set by calculating the weighted sum of values of the nodes in previous layer and applying an activation function to that weighted sum.

Multiclass Neural Network Configuration in MS Visual Studio

1. Add the *Two-Class Neural Network* module to the experiment.
2. Specify how you want the model to be trained, by setting the Create trainer mode option.
 - a. *Single Parameter*:
If you know how you want to configure the model, provide a specific set of values as arguments.
 - b. *Parameter Range*:
If unsure of best parameters, find optimal parameters by specifying multiple values and using the *TMh* module to find an optimal configuration. The trainer will iterate over multiple combinations of the settings provided and determine the combination of values that produces the best model.
3. For *Hidden layer specification*, select the type of network architecture to create.
 - a. *Fully-connected case*.
Choose this option to use the default neural network architecture.
The default multi-class neural network model is defined as follows:
 - i. The neural network model has one hidden layer.
 - ii. The output layer is fully connected to the hidden layer, which is fully connected to the input layer.
 - iii. The number of nodes in the input layer is determined by the number of features in training data.
 - iv. The number of nodes in the hidden layer is determined by the user (with a default value of 100).
 - v. The number of nodes in the output layer depends on the number of classes.
 - b. *Custom definition script*.
Choose this option to create a custom NN architecture, using the Net# language to define the number of hidden layers, connections and advanced options such as specifying the mappings between layers. After selecting the custom script option, use the *NN definition* text box to enter statements written in Net#.
4. For *Number of hidden nodes*, type the number of hidden nodes. Default is one hidden layer with 100 nodes.
5. For *The learning rate*, define the size of the step taken at each iteration, before correction.
A larger value for learning rate can cause the model to converge faster, but it can overshoot local minima.
6. For *Number of learning iterations*, specify max times number the algorithm should process training cases.
7. For *The initial learning weights diameter*, specify node weights at the start of the learning process.
8. For *The momentum*, specify a weight to apply during learning to nodes from previous iterations
9. In *The type of normalizer*, select method to use for normalization. The normalization methods supported are:
 - a. *Binning normalizer*.
It creates bins of equal size to normalize every value in each bin to be divided by total number of bins.

- b. *Gaussian normalizer*.
It rescales the values of each feature to have mean 0 and variance 1. This is done by computing the mean and the variance of each feature, and then, for each instance, subtracting the mean value and dividing by the square root of the variance (the standard deviation).
 - c. *Min-max normalizer*.
It linearly rescales every feature to the [0,1] interval. The rescaling is done by shifting the values of each feature so that the minimal value is 0, and then dividing by the new maximal value (which is the difference between the original maximal and minimal values).
 - d. *Do not normalize*.
No normalization is performed.
10. Select the *Shuffle examples* option to shuffle cases between iterations.
If deselected, cases are processed in exactly the same order each time the experiment is ran.
 11. For *Random number seed*, type a value to use as the seed.
This is useful when you want to ensure repeatability across runs of the same experiment.
 12. Select the *Allow unknown categorical levels* option to create a grouping for Unknown values in the training and validation sets. If deselected, the model can accept only the values contained in the training data. In the former case, the model might be less precise on known values but provide better predictions for new values.
 13. Train the model.
 - a. Create *trainer mode* to *Single Parameter Set*, connect a tagged dataset and the Train Model module.
 - b. Create *trainer mode* to *Parameter Range Set*, connect a tagged dataset and train the model by using *TMH*

Note: passing a parameter range to Train Model, it will use only the first value in parameter range list.

 - i. Passing a single set of parameter values to the TMH module, when it expects a range of settings for each parameter, it ignores the values and using default values for the learner.
 - ii. Selecting Parameter Range option and enter a single value for any parameter, that single given value will be used throughout the sweep, even if other parameters change across a range of values
 14. When the model is trained, right-click the output of the *Train Model* module, or *TMH* module, and select *Visualize* to see a summary of the model's parameters, together with the feature weights learned from the training, and other parameters of the NN.

Custom Definition Script in MS Visual Studio

1. A NN model is defined by the structure of its graph, which includes these attributes:
 - Number of hidden layers
 - Number of nodes in each hidden layer
 - Connections between the layers
 - Choice of activation functions
 - Weights on the graph edges
2. The structure of the graph and the activation function are determined by the user. The weights on the edges are learned when training the NN on the input data.
 - The first layer is always the input layer.
 - Last layer is always the output layer. Nodes number in output layer should be equal to the classes number.
 - Any number of intermediate layers can be defined
3. The architecture of NN models created can be changed by using the Net# language:
 - Creating hidden layers and controlling the number of nodes in each layer.
 - Specifying how layers are to be connected to each other.
 - Defining special connectivity structures, such as convolutions and weight sharing bundles.
 - Specifying different activation functions.

- Software

Software Specification for C# in MS Visual Studio

Net# is a language developed by MS used to define NN architectures for NN modules in MS Azure Machine Learning. MS provide the following information to implement with NN with this with Net#.

1. Basic concepts related to neural networks

2. Neural network requirements and how to define the primary components

3. The syntax and keywords of the Net# specification language

4. Examples of custom neural networks created using Net#

1. Neural Network(NN) basics

A NN structure consists of *nodes* organized in *layers*, and weighted *connections* between the nodes. The connections are directional, and each edge has a *source* and *destination* node. Each *trainable layer* (hidden or output layer) has one or more *connection bundles*. A connection bundle consists of a source layer and a specification of its connections. All the connections in a given bundle share the same *source layer* and *destination layer*. In Net#, a connection bundle is considered as belonging to the bundle's destination layer.

Net# supports various kinds of connection bundles to customize the way inputs are mapped to hidden layers and mapped to the outputs. The standard bundle is a *full bundle*, in which each node in the source layer is connected to every node in the destination layer. Net# supports the following advanced connection bundles:

- *Filtered bundles*.

The user can define a predicate using the locations of the source and destination layer nodes. The nodes are connected whenever the predicate is True.

- *Convolutional bundles*.

User can define small neighborhoods of the nodes in the source layer. Each node in the destination layer is connected to one neighborhood of the nodes in the source layer.

- *Pooling bundles* and *Response normalization bundles*.

These are similar to convolutional bundles in that the user defines small neighborhoods of the nodes in the source layer. The difference is that the weights of the edges in these bundles are not trainable. Instead, a predefined function is applied to the source node values to determine the destination node value.

Using Net# to define the structure of a NN makes it possible to define complex structures such as deep neural networks or convolutions of arbitrary dimensions, which are known to improve learning on data.

2. Supported customizations

The NN architecture models created in Azure Machine Learning can be extensively customized by Net#:

- Create hidden layers and control the number of nodes in each layer.
- Specify how layers are to be connected to each other.
- Define special connectivity structures, such as convolutions and weight sharing bundles.
- Specify different activation functions.

General requirements

- There must be exactly one output layer, at least one input layer, and zero or more hidden layers.
- Each layer has fixed number of nodes, conceptually sorted in a rectangular array of arbitrary dimensions.
- Input layers have no associated trained parameters and represent where instance data enters the network.
- Trainable layers (hidden and output layers) have associated trained parameters, weights and biases.
- The source and destination nodes must be in separate layers.
- Connections must be acyclic - there cannot be a chain of edges leading back to the initial source node.
- The output layer cannot be a source layer of a connection bundle.

3. Structure specifications

A NN structure specification is composed of three sections: the *constant declaration*, *layer declaration*, *connection declaration*, and also an optional *share declaration* section. These can be specified in any order.

Constant declaration (optional)

It provides a means to define values used elsewhere in the NN definition. The declaration statement consists of an identifier followed by an equal sign and a value expression.

- For example, the following statement defines a constant x: `Const X = 28;`
- To define two or more constants simultaneously, enclose the identifier names and values in braces, and separate them by using semicolons. For example: `Const {X = 28; Y = 4; }`
- The right-hand side of each assignment expression can be an integer, a real number, a Boolean value (True or False), or a mathematical expression. For example: `Const { X = 17 * 2; Y = true; }`

Layer declaration (required)

It defines the size and source of the layer, including its connection bundles and attributes. The declaration statement starts with the name of the layer (input, hidden, or output), followed by the dimensions of the layer (a tuple of positive integers).

- For example

```
input Data auto;
hidden Hidden [5,20] from Data all; | output Result [2] from Hidden all;
```
- The product of the dimensions is the number of nodes in the layer. In the example above, there are two dimensions [5,20], which means there are 100 nodes in the layer.
- The layers can be declared in any order, with one exception: If more than one input layer is defined, the order in which they are declared must match the order of features in the input data.

To specify that the number of nodes in a layer be determined automatically, use the *auto* keyword.

The *auto* keyword has different effects, depending on the layer:

- In an input layer declaration, the number of nodes is the number of features in the input data.
- In a hidden layer declaration, number of nodes is specified by the *Number of hidden nodes*.
- In an output layer declaration, the number of nodes is 2 for two-class classification, 1 for regression, and equal to the number of output nodes for multiclass classification.

For example, the following network definition allows the size of all layers to be automatically determined:

```
input Data auto;
hidden Hidden auto from Data all; | output Result auto from Hidden all;
```

A layer declaration for a trainable layer (hidden or output layers) could include the output function (also called an activation function), which defaults to *sigmoid* for classification models, and *linear* for regression models. (Even if the default is used, the activation function can be explicitly stated, if desired for clarity.)

The following output functions are supported:

- | | | | | |
|-----------|-----------|----------|------------|------------|
| • sigmoid | • softmax | • square | • srlinear | • tanh |
| • linear | • rlinear | • sqrt | • abs | • brlinear |

For example, the following declaration uses the *softmax* function:

```
output Result [100] softmax from Hidden all;
```

Connection declaration

Immediately after defining the trainable layer, connections among the layers defined must be declared. The connection bundle declaration starts with the keyword *from*, followed by the name of the bundle's source layer and the kind of connection bundle to create. Currently, five kinds of connection bundles are supported:

- *Full* bundles, indicated by the keyword *all*
- *Filtered* bundles, indicated by the keyword *where*, followed by a predicate expression
- *Convolutional* bundles, indicated by the keyword *convolve*, followed by the convolution attributes
- *Pooling* bundles, indicated by the keywords *max pool* or *mean pool*
- *Response normalization* bundles, indicated by the keyword *response norm*

Full bundles

A full connection bundle includes a connection from each node in the source layer to each node in the destination layer. This is the default network connection type.

Filtered bundles

A filtered connection bundle specification includes a predicate, expressed syntactically, much like a C# lambda expression. The following example defines two filtered bundles:

```
input Pixels [10, 20];
```

```
hidden ByRow[10, 12] from Pixels where (s, d) => s[0] == d[0];
```

```
hidden ByCol[5, 20] from Pixels where (s, d) => abs(s[1] - d[1]) <= 1;
```

- In the predicate for *ByRow*, *s* is a parameter representing an index into the rectangular array of nodes of the input layer, *Pixels*, and *d* is a parameter representing an index into the array of nodes of the hidden layer, *ByRow*. The type of both *s* and *d* is a tuple of integers of length two. Conceptually, *s* ranges over all pairs of integers with $0 \leq s[0] < 10$ and $0 \leq s[1] < 20$, and *d* ranges over all pairs of integers, with $0 \leq d[0] < 10$ and $0 \leq d[1] < 12$.
- On the right-hand side of the predicate expression, there is a condition. In this example, for every value of *s* and *d* such that the condition is True, there is an edge from the source layer node to the destination layer node. Thus, this filter expression indicates that the bundle includes a connection from the node defined by *s* to the node defined by *d* in all cases where *s*[0] is equal to *d*[0].

Optionally, you can specify a set of weights for a filtered bundle. The value for the *Weights* attribute must be a tuple of floating point values with a length that matches the number of connections defined by the bundle. By default, weights are randomly generated. Weight values are grouped by the destination node index. That is, if the first destination node is connected to *K* source nodes, the first *K* elements of the *Weights* tuple are the weights for the first destination node, in source index order. The same applies for the remaining destination nodes. It's possible to specify weights directly as constant values. For example, if you learned the weights previously, you can specify them as constants using this syntax:

```
const Weights_1 = [0.0188045055, 0.130500451, ...]
```

Convolutional bundles

When the training data has a homogeneous structure, convolutional connections are commonly used to learn high-level features of the data. Convolutional bundles employ rectangular *kernels* that are slid through the dimensions. Essentially, each kernel defines a set of weights applied in local neighborhoods, referred to as *kernel applications*. Each kernel application corresponds to a node in the source layer, which is referred to as the *central node*. The weights of a kernel are shared among many connections. In a convolutional bundle, each kernel is rectangular and all kernel applications are the same size. Convolutional bundles support the following attributes:

InputShape defines the dimensionality of the source layer for the purposes of this convolutional bundle. The value must be a tuple of positive integers. The product of the integers must equal the number of nodes in the source layer, but otherwise, it does not need to match the dimensionality declared for the source layer. The length of this tuple becomes the *arity* value for the convolutional bundle. (Typically arity refers to the number of arguments or operands that a function can take.) To define kernels shape and locations use attributes *KernelShape*, *Stride*, *Padding*, *LowerPad*, & *UpperPad*:

- *KernelShape*: (required) Defines the dimensionality of each kernel for the convolutional bundle. The value must be a tuple of positive integers with a length that equals the arity of the bundle. Each component of this tuple must be no greater than the corresponding component of *InputShape*.
- *Stride*: (optional) Defines the sliding step sizes of the convolution (one step size for each dimension), that is the distance between the central nodes. The value must be a tuple of positive integers with a length that is the arity of the bundle. Each component of this tuple must be no greater than the corresponding component of *KernelShape*. Default value is a tuple with all components equal to one.
- *Sharing*: (optional) Defines the weight sharing for each dimension of the convolution. The value can be a single Boolean value or a tuple of Boolean values with a length that is the arity of the bundle. A single Boolean value is extended to be a tuple of the correct length with all components equal to the specified value. The default value is a tuple that consists of all True values.

- *MapCount*: (optional) Defines the number of feature maps for the convolutional bundle. The value can be a single positive integer or a tuple of positive integers with a length that is the arity of the bundle. A single integer value is extended to be a tuple of the correct length with the first components equal to the specified value and all the remaining components equal to one. The default value is one. The total number of feature maps is the product of the components of the tuple. The factoring of this total number across the components determines how the feature map values are grouped in the destination nodes.
- *Weights*: (optional) Defines the initial weights for the bundle. The value must be a tuple of floating point values with a length that is the number of kernels times the number of weights per kernel, as defined later in this article. The default weights are randomly generated.

There are two sets of properties that control padding, the properties being mutually exclusive:

- *Padding*: (optional) Determines whether the input should be padded by using a *default padding scheme*. The value can be a single Boolean value, or it can be a tuple of Boolean values with a length that is the arity of the bundle. A single Boolean value is extended to be a tuple of the correct length with all components equal to the specified value. If the value for a dimension is True, the source is logically padded in that dimension with zero-valued cells to support additional kernel applications, such that the central nodes of the first and last kernels in that dimension are the first and last nodes in that dimension in the source layer. Thus, the number of "dummy" nodes in each dimension is determined automatically, to fit exactly $(InputShape[d] - 1) / Stride[d] + 1$ kernels into the padded source layer. If value for a dimension is False, the kernels are defined so that the number of nodes on each side that are left out is the same (up to a difference of 1). Default attribute's value is a tuple with all components equal to False.
- *UpperPad* and *LowerPad*: (optional) Provide greater control over the amount of padding to use. *Important*: These attributes can be defined if and only if the *Padding* property above is *not* defined. The values should be integer-valued tuples with lengths that are the arity of the bundle. When these attributes are specified, "dummy" nodes are added to the lower and upper ends of each dimension of the input layer. The number of nodes added to the lower and upper ends in each dimension is determined by *LowerPad[i]* and *UpperPad[i]* respectively. To ensure that kernels correspond only to "real" nodes and not to "dummy" nodes, the following conditions must be met:
 - Each component of *LowerPad* must be strictly less than $KernelShape[d]/2$.
 - Each component of *UpperPad* must be no greater than $KernelShape[d]/2$.
 - The default value of these attributes is a tuple with all components equal to 0.

The setting *Padding* = true allows as much padding as is needed to keep the "center" of the kernel inside the "real" input. This changes the math a bit to compute the output size. Generally, the output size D is computed as $D = (I - K)/S + 1$, where I is the input size, K is the kernel size, S is the stride, and $/$ is integer division (round toward zero). If you set *UpperPad* = [1, 1], the input size I is effectively 29, and thus $D = (29 - 5)/2 + 1 = 13$. However, when *Padding* = true, essentially I gets bumped up by $K - 1$; hence $D = ((28 + 4) - 5)/2 + 1 = 27/2 + 1 = 13 + 1 = 14$. By specifying values for *UpperPad* and *LowerPad* you get much more control over the padding than if you just set *Padding* = true.

Pooling bundles

A *pooling bundle* applies geometry similar to convolutional connectivity, but it uses predefined functions to source node values to derive the destination node value. Hence, pooling bundles have no trainable state (weights or biases). Pooling bundles support all the convolutional attributes except *Sharing*, *MapCount*, and *Weights*. Typically, the kernels summarized by adjacent pooling units do not overlap. If *Stride[d]* is equal to *KernelShape[d]* in each dimension, the layer obtained is the traditional local pooling layer, which is commonly employed in convolutional neural networks. Each destination node computes the maximum or the mean of the activities of its kernel in the source layer. The following illustrates a pooling bundle:

<pre>hidden P1 [5, 12, 12] from C1 max pool { InputShape = [5, 24, 24];</pre>	<pre>KernelShape = [1, 2, 2]; Stride = [1, 2, 2]; }</pre>
---	---

- The arity of the bundle is 3 (the length of the tuples *InputShape*, *KernelShape*, and *Stride*).
- The number of nodes in the source layer is $5 * 24 * 24 = 2880$.
- This is a traditional local pooling layer because *KernelShape* and *Stride* are equal.
- The number of nodes in the destination layer is $5 * 12 * 12 = 1440$.

Response normalization bundles (RNB)

Response normalization is a local normalization scheme that was first introduced by Geoffrey Hinton. This is used to aid generalization in neural nets. When one neuron is firing at a very high activation level, a local response normalization layer suppresses the activation level of the surrounding neurons. This is done by using three parameters (α , β , and k) and a convolutional structure (or neighborhood shape). Every neuron in the destination layer y corresponds to a neuron x in the source layer. The activation level of y is given by the following formula, where f is the activation level of a neuron, and N_x is the kernel (or the set that contains the neurons in the neighborhood of x), as defined by the following convolutional structure:

$$\frac{f(x)}{\left(k + \frac{\alpha}{|N_x|} \sum_{z \in N_x} (f(z))^2\right)^\beta}$$

RNB support all convolutional attributes except *Sharing*, *MapCount*, and *Weights*.

- If the kernel contains neurons in the same map as x , the normalization scheme is referred to as *same map normalization* – to define it, the first coordinate in *InputShape* must have the value 1.
- If the kernel contains neurons in the same spatial position as x , but the neurons are in other maps, the normalization scheme is called *across maps normalization*. This type of response normalization implements a form of lateral inhibition inspired by the type found in real neurons, creating competition for big activation levels amongst neuron outputs computed on different maps. To define across maps normalization, the first coordinate must be an integer greater than one and no greater than the number of maps, and the rest of the coordinates must have the value 1.

Because response normalization bundles apply a predefined function to source node values to determine the destination node value, they have no trainable state (weights or biases).

Alert: The nodes in the destination layer correspond to neurons that are the central nodes of the kernels.

For example;

- if *KernelShape[d]* is odd, then *KernelShape[d]/2* corresponds to the central kernel node.
- If *KernelShape[d]* is even, the central node is at *KernelShape[d]/2 - 1*.
- Therefore, if *Padding[d]* is False, the first and the last *KernelShape[d]/2* nodes do not have corresponding nodes in the destination layer.

To avoid this situation, define *Padding* as *[true, true, ..., true]*. In addition to the four attributes described earlier, response normalization bundles also support the following attributes:

- *Alpha*: (required) Specifies a floating-point value that corresponds to α in the previous formula.
- *Beta*: (required) Specifies a floating-point value that corresponds to β in the previous formula.
- *Offset*: Specifies a floating-point value that corresponds to k in the previous formula. It defaults to 1.

The following example defines a response normalization bundle using these attributes:

```
hidden RN1 [5, 10, 10]
from P1 response norm {
  InputShape = [ 5, 12, 12];
  KernelShape = [ 1, 3, 3];
  Alpha = 0.001;
  Beta = 0.75;
}
```

- The source layer includes five maps, each with a of dimension of 12x12, totaling in 1440 nodes.
- The value of *KernelShape* indicates this is a same map normalization layer, where neighborhood is a 3x3 rectangle.
- The default value of *Padding* is False, thus the destination layer has only 10 nodes in each dimension. To include one node in the destination layer that corresponds to every node in the source layer, add *Padding* = [true, true, true]; and change the size of RN1 to [5, 12, 12].

Share declaration

Net# optionally supports defining multiple bundles with shared weights. The weights of any two bundles can be shared if their structures are the same. The following syntax defines bundles with shared weights:

• share-declaration: share {layer-list} share {bundle-list} share {bias-list} • layer-list: layer-name, layer-name layer-list, layer-name	• bundle-list: bundle-spec, bundle-spec bundle-list, bundle-spec • bundle-spec: layer-name => layer-name	• bias-list: bias-spec, bias-spec bias-list, bias-spec • bias-spec: 1=> layer-name • layer-name: identifier
--	--	--

The following share-declaration specifies layer names, indicating both weights and biases should be shared:

Const { InputSize = 37; HiddenSize = 50; } input { Data1 [InputSize]; Data2 [InputSize]; }	hidden { H1 [HiddenSize] from Data1 all; H2 [HiddenSize] from Data2 all; }	output Result [2] { from H1 all; from H2 all; } share {H1, H2} // share both weights and biases
---	---	---

- The input features are partitioned into two equal sized input layers.
- The hidden layers then compute higher level features on the two input layers.
- The share-declaration specifies that *H1* and *H2* must be computed the same way from respective inputs.

Alternatively, this could be specified with two separate share-declarations as follows:

- share {Data1 => H1, Data2 => H2} // share weights
- share {1 => H1, 1 => H2} // share biases

You can use the short form only when the layers contain a single bundle. In general, sharing is possible only when the relevant structure is identical (have the same size), same convolutional geometry, and so forth.

Examples of Net# usage

This are some examples of how to use Net# to add hidden layers, define the way that hidden layers interact with other layers, and build convolutional networks.

1) Define a simple custom neural network: "Hello World" example

This simple example demonstrates how to create a neural network model that has a single hidden layer.

```
input Data auto;      | hidden H [200] from Data all;      | output Out [10] sigmoid from H all;
```

The example illustrates some basic commands as follows:

- The first line defines the input layer (named *Data*). When you use the *auto* keyword, the neural network automatically includes all feature columns in the input examples.
- The second line creates the hidden layer. The name *H* is assigned to the hidden layer, which has 200 nodes. This layer is fully connected to the input layer.
- The third line defines the output layer (named *O*), which contains 10 output nodes. If the neural network is used for classification, there is one output node per class. The keyword *sigmoid* indicates that the output function is applied to the output layer.

2) Define multiple hidden layers: computer vision example

This demonstrates how to define a more complex neural network, with multiple custom hidden layers.

```
// Define the input layers | // Define the first two hidden layers, using data only from the Pixels input
input Pixels [10, 20];     | hidden ByRow [10, 12] from Pixels where (s,d) => s[0] == d[0];
input MetaData [7];        | hidden ByCol [5, 20] from Pixels where (s,d) => abs(s[1] - d[1]) <= 1;
```

```
// Define the third hidden layer, which uses as source the hidden layers ByRow and ByCol
hidden Gather [100]
{
  from ByRow all;
  from ByCol all;
}

// Define the output layer and its sources
output Result [10]
{
  from Gather all;
  from MetaData all;
}
```

This example illustrates several features of the neural networks specification language:

- The structure has two input layers, *Pixels* and *MetaData*.
- *Pixels* layer is a source layer for two connection bundles, with destination layers, *ByRow* and *ByCol*.
- The layers *Gather* and *Result* are destination layers in multiple connection bundles.
- The output layer, *Result*, is a destination layer in two connection bundles; one with the second level hidden (*Gather*) as a destination layer, and with the input layer (*MetaData*) as a destination layer.
- Hidden layers, *ByRow* and *ByCol*, specify filtered connectivity by using predicate expressions. More precisely, the node in *ByRow* at $[x, y]$ is connected to nodes in *Pixels* that have the first index coordinate equal to the node's first coordinate, x . Node in *ByCol* at $[x, y]$ is connected to the nodes in *Pixels* having the second index coordinate within one of the node's second coordinate, y .

3) Define a convolutional network for multiclass classification: digit recognition example

The is designed to recognize numbers and illustrates advanced techniques to customize a neural network.

```
input Image [29, 29];
hidden Conv1 [5, 13, 13] from Image convolve
{
  InputShape = [29, 29];
  KernelShape = [ 5, 5];
  Stride      = [ 2, 2];
  MapCount    = 5;
}
hidden Conv2 [50, 5, 5]
from Conv1 convolve

{
  InputShape = [ 5, 13, 13];
  KernelShape = [ 1, 5, 5];
  Stride      = [ 1, 2, 2];
  Sharing     = [false, true, true];
  MapCount    = 10;
}
hidden Hid3 [100] from Conv2 all;
output Digit [10] from Hid3 all;
```

- The structure has a single input layer, *Image*.
- The keyword *convolve* indicates that the layers named *Conv1* and *Conv2* are convolutional layers. Each of these layer declarations is followed by a list of the convolution attributes.
- The net has a third hidden layer, *Hid3*, which is fully connected to the second hidden layer, *Conv2*.
- The output layer, *Digit*, is connected only to the third hidden layer, *Hid3*. The keyword *all* indicates that the output layer is fully connected to *Hid3*.
- The arity of the convolution is three (tuples length *InputShape*, *KernelShape*, *Stride*, and *Sharing*).
- The number of weights per kernel is $1 + \text{KernelShape}[0] * \text{KernelShape}[1] * \text{KernelShape}[2] = 1 + 1 * 5 * 5 = 26$. Or $26 * 50 = 1300$.
- You can calculate the nodes in each hidden layer as follows:
 - $\text{NodeCount}[0] = (5 - 1) / 1 + 1 = 5$.
 - $\text{NodeCount}[1] = (13 - 5) / 2 + 1 = 5$.
 - $\text{NodeCount}[2] = (13 - 5) / 2 + 1 = 5$.
- The total number of nodes can be calculated by using the declared dimensionality of the layer, $[50, 5, 5]$, as follows: $\text{MapCount} * \text{NodeCount}[0] * \text{NodeCount}[1] * \text{NodeCount}[2] = 10 * 5 * 5 * 5$
- Because *Sharing*[d] is False only for $d == 0$, the number of kernels is $\text{MapCount} * \text{NodeCount}[0] = 10 * 5 = 50$.